

目录

1 一笔 PCIe Memory Read，到底是怎么完成的？

- 1.1 先看整条路径：为什么读比写多了一个“返回数据”的过程
- 1.2 第一步：CPU 发起 Load，本质上仍然是一次本地地址访问
- 1.3 第二步：Outbound 地址转换，把本地地址翻译成 PCIe 地址
- 1.4 第三步：Controller 生成一笔 MRd
- 1.5 为什么读比写复杂得多？
- 1.6 第四步：Endpoint 收到 MRd 后，先 BAR 命中，再去本地取数
- 1.7 第五步：对端拿到数据之后，要把它装进 CplD
- 1.8 Requester / Completer 不是固定角色，而是这笔事务里的角色
- 1.9 CplD 返回时，系统靠什么找到原始 MRd？
- 1.10 为什么一笔 MRd 可能对应多个 CplD？
- 1.11 最后一步：RC 匹配完成后，数据怎样回到 CPU？
- 1.12 为什么 Memory Read 会超时？
 - 1.12.1 CPU / AXI 真的发起了 Read 吗？
 - 1.12.2 NoC / AXI 是否命中 PCIe Window？
 - 1.12.3 Outbound ATU 是否命中？
 - 1.12.4 RC 真的发出了 MRd 吗？
 - 1.12.5 Endpoint 是否收到了 MRd，并正确 BAR 命中？
 - 1.12.6 Endpoint 本地 AXI Read 是否真正完成？
 - 1.12.7 CplD 是否被正确返回并匹配？
- 1.13 最容易理解错的 6 个点
- 1.14 误区 1：MRd 发出去时，数据已经在包里了
- 1.15 误区 2：CplD 只靠顺序匹配
- 1.16 误区 3：RC 永远是 Requester
- 1.17 误区 4：CplD 只会有一个
- 1.18 误区 5：只要链路是 L0，读就一定不会超时
- 1.19 误区 6：Memory Read 和 Memory Write 只是“一个读、一个写”而已
- 1.20 一张图记住全文
- 1.21 后记
- 1.22 《PCIe Tag 到底有什么用？为什么 Read 可以乱序返回？》
- 1.23 参考资料

2 PCIe Tag 到底有什么用？为什么 Read 可以乱序返回？

- 2.1 Tag 到底在解决什么问题？
- 2.2 如果只有一笔 MRd，其实根本感受不到 Tag 的存在
- 2.3 先看最直观的问题：为什么不能只靠“返回顺序”？
- 2.4 为什么 PCIe 允许 Read 乱序返回？

- 2.5 Tag 的第一层作用：给每一笔未完成请求一个编号
- 2.6 光有 Tag 还不够，为什么还要有 Requester ID？
- 2.7 Outstanding Request Table：Tag 真正落地的地方
- 2.8 Tag 的第二层作用：支持高并发读
- 2.9 Tag 的生命周期：从空闲到释放
- 2.10 为什么一笔 MRd 可能不止一个 CplID？
- 2.11 Tag 满了，是不是 PCIe 出故障了？
- 2.12 正常饱和状态
- 2.13 真正异常状态
- 2.14 乱序返回 + 多个 Completion + 生命周期，构成了 Tag 的完整意义
 - 2.14.1 Tag 给每笔 Outstanding Read Request 一个编号
 - 2.14.2 Tag 让系统能支持多个并发 Read 并允许乱序返回
 - 2.14.3 Tag 必须和 Outstanding Table 一起管理
- 2.15 Debug 时，最应该围绕 Tag 查什么？
 - 2.15.1 Tag 分配是否唯一？
 - 2.15.2 Outstanding Table 是否正确建表？
 - 2.15.3 返回的 Completion 是否带回了正确的 Requester ID 和 Tag？
 - 2.15.4 如果 Completion 乱序返回，Controller 是否仍能正确匹配？
 - 2.15.5 如果一笔请求被拆成多个 CplID，Tag 是否被提前释放？
 - 2.15.6 请求完成后，表项是否正常清除？
 - 2.15.7 超时和迟到 Completion 如何处理？
- 2.16 最容易理解错的 6 个点
- 2.17 误区 1：Tag 是为了“加速链路”
- 2.18 误区 2：Tag 在整个 PCIe 系统里必须全局唯一
- 2.19 误区 3：乱序返回是一种异常行为
- 2.20 误区 4：只收到一个 Completion，说明这笔请求可以结束了
- 2.21 误区 5：Tag 满就一定是故障
- 2.22 误区 6：Tag 可以脱离 Outstanding Table 单独理解
- 2.23 一张图记住全文
- 2.24 后记
- 2.25 《PCIe Credit 到底是什么？为什么它和 Tag 完全不是一回事？》
- 2.26 参考资料
- 3 CPU 写一个地址，PCIe 到底发生了什么？
 - 3.1 先问一个很具体的问题
 - 3.2 第一步：CPU 发起的其实只是一次本地访存
 - 3.3 第二步：本地地址还不是 PCIe 总线地址
 - 3.4 第三步：AXI Write 在 Controller 里变成 Memory Write TLP
 - 3.5 第四步：为什么 Memory Write 不需要 Completion？

- 3.6 第五步：PCIe 差分线上到底传的是什么？
 - 3.7 第六步：Endpoint 收到 TLP 后，为什么要先看 BAR？
 - 3.8 最后一步：又变回一笔本地 AXI Write
 - 3.9 为什么这条路径对 Debug 特别重要？
 - 3.9.1 CPU 真的发起 Store 了吗？
 - 3.9.2 NoC / AXI 真的命中 PCIe Window 了吗？
 - 3.9.3 Outbound Translation 命中了吗？
 - 3.9.4 Controller 真的生成 MWr 了吗？
 - 3.9.5 链路真的把包发出去了么？
 - 3.9.6 Endpoint 是否 BAR Hit？
 - 3.9.7 本地 AXI / 寄存器真的写成功了吗？
 - 3.10 最容易理解错的 5 个地方
 - 3.11 误区 1：AXI 会直接跨芯片传输
 - 3.12 误区 2：PHY 知道 BAR 和地址
 - 3.13 误区 3：BAR 就是一段内存
 - 3.14 误区 4：MWr 没有 Completion，所以无法判断链路是否可靠
 - 3.15 误区 5：写失败优先查 PHY
 - 3.16 一张图记住全文
 - 3.17 后记
 - 3.18 《一笔 PCIe Memory Read，到底是怎么完成的？》
 - 3.19 参考资料
 - 4. PCIe 为什么不会把接收端 Buffer 撑爆？理解 Credit 流控机制
 - 4.1 Credit 流控解决的问题
 - 4.2 PCIe 的六类基础 Credit
 - 4.3 一笔 TLP 如何消耗 Credit
 - 4.4 Credit 如何建立和更新
 - 4.5 Credit 与 Tag 的区别
 - 4.6 可用 Credit 被耗尽时会发生什么
 - 4.7 一笔 TLP 发不出去时怎么查
 - 4.7.1 先确认 Requester 真的生成了 Request
 - 4.7.2 再确认这笔 TLP 需要哪类 Credit
 - 4.7.3 检查 Flow Control 是否已经初始化完成
 - 4.7.4 检查 Credit 是否继续更新
 - 4.7.5 有 Switch 时逐跳检查
 - 4.8 PCIe 6.0 Flit Mode 的边界
 - 4.9 一张图记住 Credit
 - 4.10 参考资料
-

1 一笔 PCIe Memory Read，到底是怎么完成的？

来源: <https://mp.weixin.qq.com/s/gytQOlu4Wvrkul8KBXGJBQ>

作者: 娃围玮未

update 2026/08/16 03 : 11

已截图

《PCIe 进阶之路》第 2 篇 · 不背协议，沿着真实数据路径学 PCIe 公众号：芯片进阶之路 · 知乎专栏：芯片设计进阶之路 · 作者：娃围玮未

在上一篇《CPU 写一个地址，PCIe 到底发生了什么？》里，我们沿着一笔 Store 走完了这样一条路径：

```
CPU Store
→ NoC / AXI
→ Outbound ATU
→ Memory Write TLP
→ PCIe Link
→ Endpoint BAR Hit
→ Inbound ATU
→ Endpoint 本地 AXI / 寄存器
```

写事务的特点很鲜明：

只要把包送到对端，正常情况下就不需要再把什么东西带回来。

但读事务不一样。

如果软件执行：

```
value = *(
volatile
volatile
unsigned
unsigned
int
int
*)
0x40001234
0x40001234
;
```

那么最终不仅要把“请求”送到对端，还要把“数据”再带回来。

也就是说，一笔 PCIe Memory Read 本质上至少包含两个阶段：

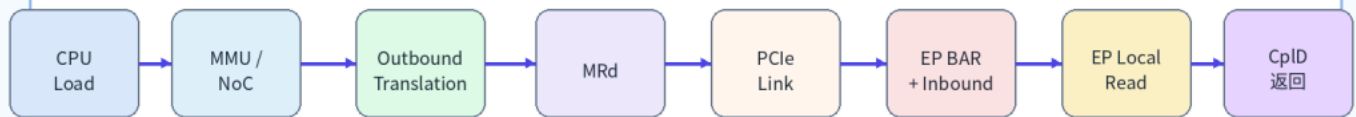
1. 发出读请求（MRd）
2. 对端把数据通过 Completion with Data（CpID）送回来

所以第二篇文章只讲一件事：

一笔 CPU Load，怎样从本地地址访问，变成一笔 MRd，再由对端返回 CpID，最终让 CPU 真正拿到数据。

一笔 PCIe Memory Read， 到底是怎么完成的？

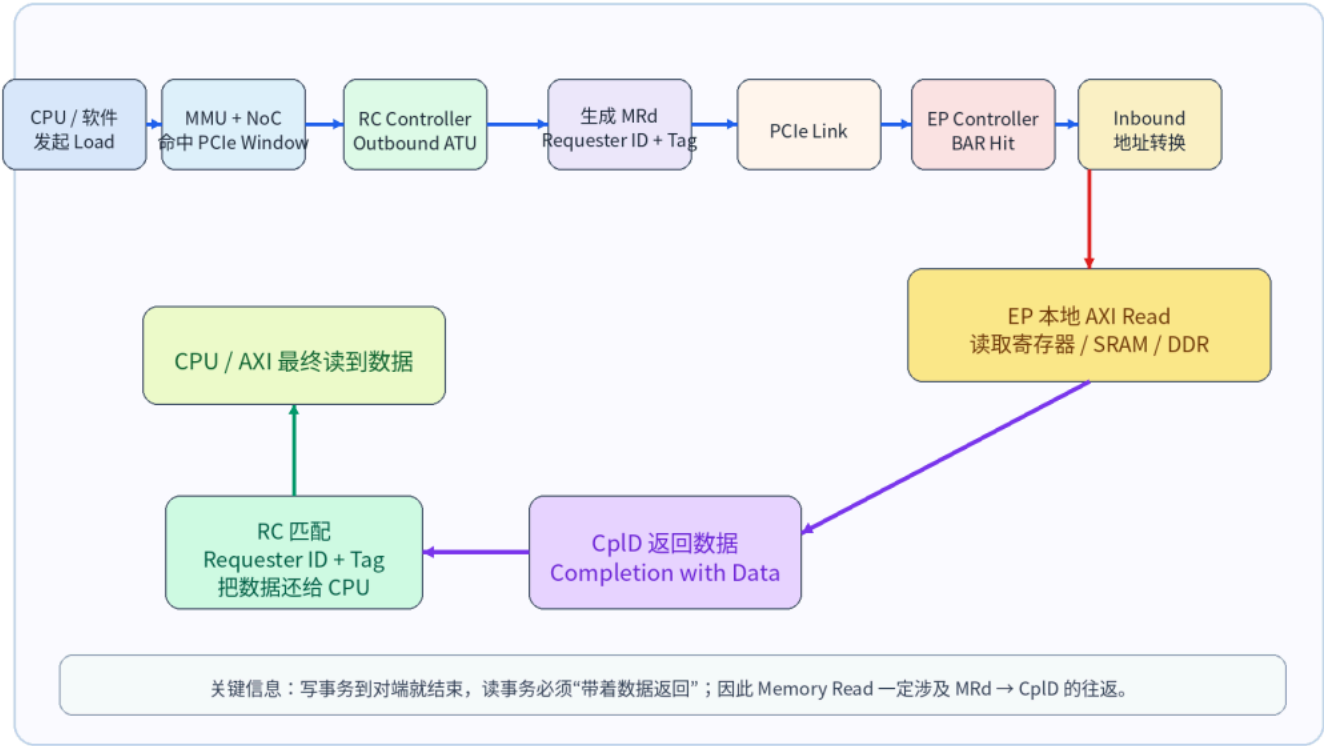
PCIe 进阶之路 · 02



Memory Read 的关键：MRd 只是“取数请求”，真正的数据必须由对端通过 CpID 返回。

1.1 先看整条路径：为什么读比写多了一个“返回数据”的过程

图 1 整体系统地图：一笔 PCIe Memory Read 的完整路径



如果把细节都先折叠，读路径可以压缩成：

```
CPU Load
↓
MMU + NoC
↓
RC Controller / Outbound ATU
↓
Memory Read TLP (MRd)
↓
PCIe Link
↓
Endpoint Controller / BAR Hit
↓
Inbound Translation
↓
Endpoint 本地 AXI Read
↓
得到读数据
↓
Completion with Data (CpID)
↓
返回 Root Complex
↓
用 Requester ID + Tag 匹配原始请求
↓
把数据交还给 CPU / AXI
```

和上一篇对比，真正新增加的部分只有一句话：

读请求必须“把数据拿回来”。

这也是为什么 Memory Read 必须比 Memory Write 多出一整套：

- Requester / Completer
- Completion
- Requester ID
- Tag
- Outstanding Request

这些概念其实并不抽象，它们只是为了解决一个现实问题：

包发出去了，但数据回来时，系统怎么知道该还给谁？

1.2 第一步：CPU 发起 Load，本质上仍然是一次本地地址访问

和写事务一样，CPU 在最开始并不知道自己要“做 PCIe”。

它只知道自己要执行一条读操作：

```
value = *(
volatile
volatile
unsigned
unsigned
int
int
*)
0x40001234
0x40001234
;
```

于是流程依旧从 SoC 内部开始：

```
CPU Virtual Address
    ↓ MMU
SoC Physical Address
    ↓ NoC / Interconnect
找到目标外设
```

如果 SoC 地址地图定义为：

```
0x4000_0000 ~ 0x4000_FFFF → PCIe Root Complex Window
```

那么这笔读操作就会被送进 PCIe Controller，而不是送去 DDR。

这一点特别值得强调，因为很多调试问题第一步就搞错了。

软件层看到的只是：

```
读地址: 0x4000_1234
```

但硬件系统真正需要回答的是：

这笔地址访问到底有没有命中 PCIe Controller?

如果连这一步都没进去，后面所有 MRd、CplD、Tag 的讨论都无从谈起。

1.3 第二步：Outbound 地址转换，把本地地址翻译成 PCIe 地址

进入 RC Controller 后，本地 SoC 地址还需要被翻译成 PCIe 总线地址。

这一步和上一篇完全一样，仍由某种 **Outbound Address Translation** 逻辑来完成。

例如：

```
RC 本地 PCIe Window: 0x4000_0000 ~ 0x4000_FFFF
Outbound Target      : 0x8000_0000
EP BAR0              : 0x8000_0000, 大小 64 KB
EP Local Target      : 0x2000_0000
```

如果 CPU 读：

```
0x4000_1234
```

那么 RC 侧转换后，会得到：

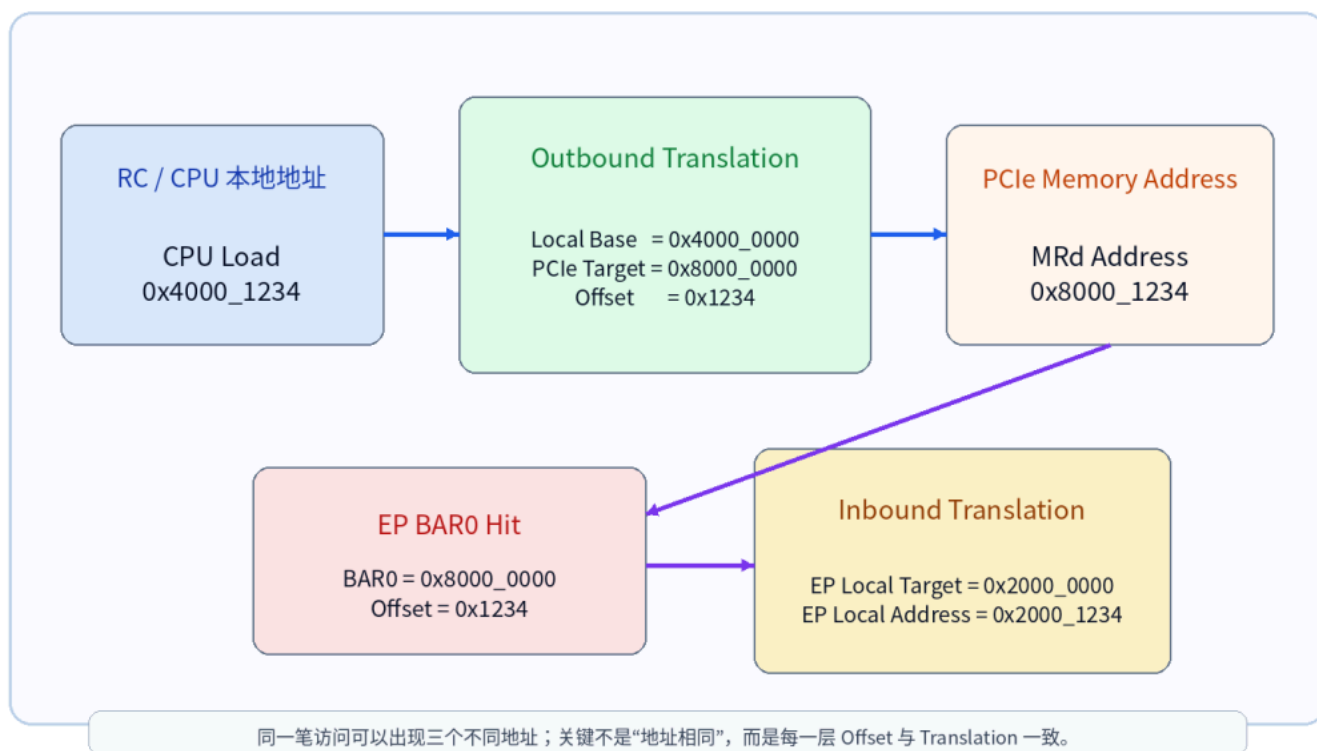
```
PCIe Address
= 0x8000_0000 + (0x4000_1234 - 0x4000_0000)
= 0x8000_1234
```

这说明：

```
CPU / AXI Address = 0x4000_1234
PCIe Address      = 0x8000_1234
```

不是一个地址概念。

图 2 地址路径的本质：本地地址 → PCIe 地址 → Endpoint 本地地址



所以当你看到波形里看到 TLP Address 是 0x8000_1234 时，不要以为软件写错了。

很可能这是完全正确的，因为：

软件地址只是本地视图，PCIe TLP 里的地址是总线视图。

1.4 第三步：Controller 生成一笔 MRd

地址转换完成后，Controller 会把这笔本地读请求转换成 PCIe Transaction Layer Packet。

这里对应的事务类型就是：

Memory Read TLP
简称：MRd

和 MWr 最大的差别在于：

- MWr 自己就带着要写的数据
- MRd 只是在说：“请把这段地址上的数据给我”

所以你可以粗略理解成：

```
AXI Read Request
Address = 0x4000_1234
↓
Outbound Translation
↓
PCIe MRd
Address = 0x8000_1234
```

这里必须建立一个核心直觉：

MRd 发出去的时候，读数据其实还不在这笔包里。

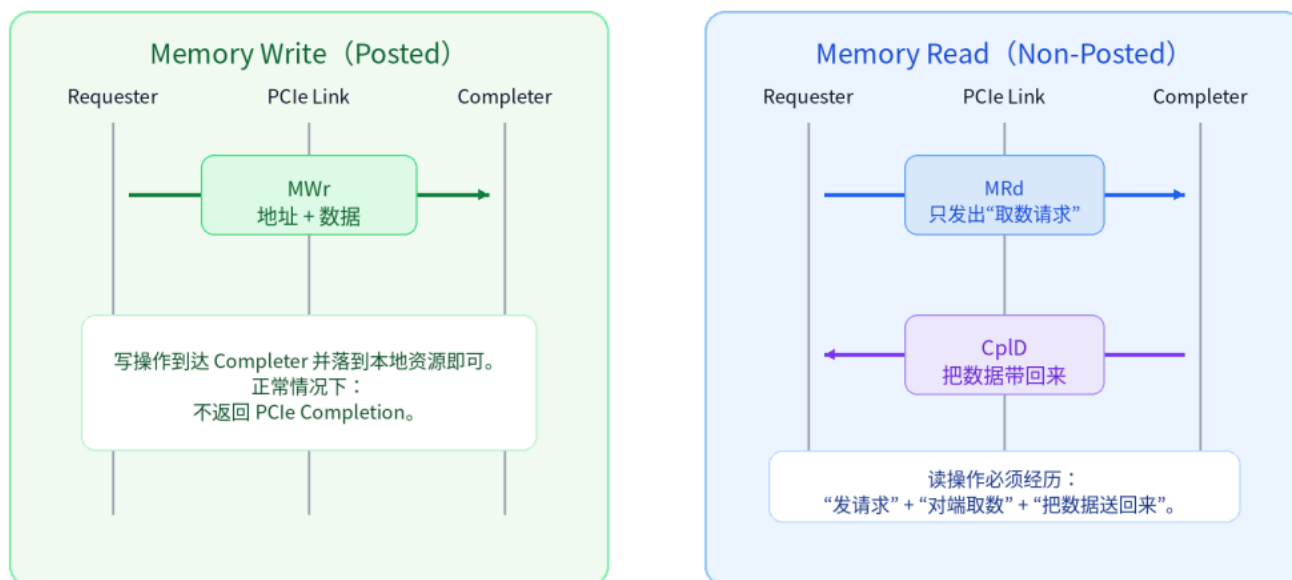
它只是一张“取数单”。

真正的数据，要等到对端完成本地读取后，再通过 Completion with Data 送回来。

1.5 为什么读比写复杂得多？

这一点用一张图就足够直观。

图 3 为什么读比写复杂：MWr 和 MRd 的路径差异



Memory Write:

```
Requester ————— MWr —————> Completer
```

正常情况下，包送到对端并落地到本地资源，写事务就结束了。

Memory Read:

```
Requester ————— MRd —————> Completer
Requester <———— CplD ————— Completer
```

所以读路径里多出来的并不是“某一个小字段”，而是整个返回路径：

```
对端取数
→ 生成 Completion
→ 把数据送回
→ 本端匹配原始请求
→ 最终把数据还给 CPU
```

这就是为什么几乎所有复杂的 PCIe 事务机制——比如 Requester ID、Tag、Outstanding——都会优先出现在 Read 路径里。

1.6 第四步：Endpoint 收到 MRd 后，先 BAR 命中，再去本地取数

MRd 到达 Endpoint 后，对端 Controller 看到的是：

```
这是一个 Memory Read 请求
目标地址 = 0x8000_1234
```

接下来它必须先确认：

这段地址是不是属于我这个 Function 暴露的 BAR 空间？

假设：

```
BAR0 Base = 0x8000_0000
BAR0 Size = 64 KB
```

那么：

```
0x8000_1234
```

落在 BAR0 范围内，于是地址命中成功。

然后 Controller 再做 Inbound 地址转换：

```
Offset = 0x8000_1234 - 0x8000_0000 = 0x1234
EP Local AXI Address = 0x2000_0000 + 0x1234 = 0x2000_1234
```

这时，对端内部最终发起的才是一笔本地 AXI Read：

```
AXI Read
Address = 0x2000_1234
```

从这里开始，对端要么去读寄存器，要么去读 SRAM，要么去读 DDR，取决于本地地址映射到了什么资源。

所以需要记住一个顺序：

```
先 BAR Hit
→ 再 Inbound Translation
→ 再本地 AXI Read
```

不是一上来就把 TLP 地址直接丢给本地 AXI。

1.7 第五步：对端拿到数据之后，要把它装进 CplD

Endpoint 的本地 AXI Read 完成后，对端终于拿到了真正的读数据。

但它还不能直接把本地 AXI 的 `RDATA` 原样扔回 Root Complex。

它需要重新封装成一笔 PCIe Completion：

```
Completion with Data
简称：CplD
```

于是读路径变成：

```
MRd 到达 Endpoint
→ EP 本地 AXI Read
→ 取到数据
→ 生成 CplD
→ CplD 返回给 RC
```

这时候 CplD 里携带的就不仅是“这是一个响应”，还包括：

- 这是谁的响应
- 响应的是哪笔请求
- 带回多少数据

- 读到的实际数据值

到这里，一个新问题自然就来了：

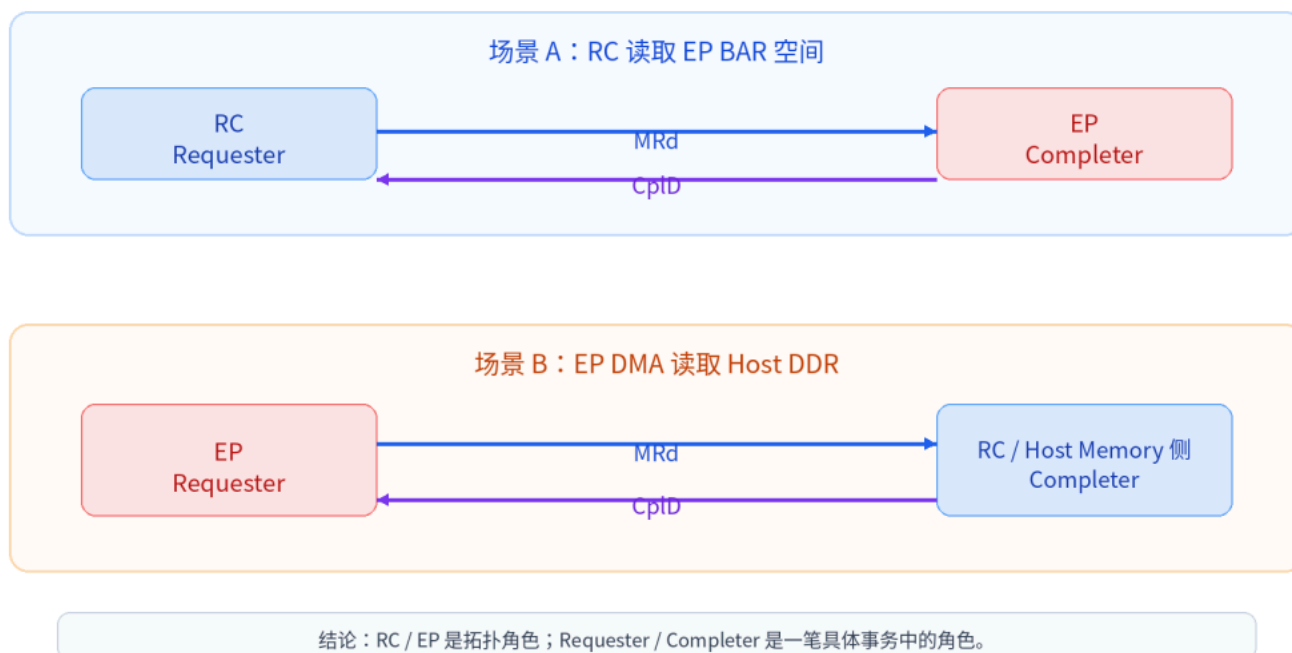
如果同时发出了很多个 MRd，返回的 CpID 怎么知道自己属于哪一笔？

这就轮到 **Requester ID 和 Tag** 出场了。

1.8 Requester / Completer 不是固定角色，而是这笔事务里的角色

在继续讲 Tag 之前，先把一个经常被误解的概念讲清楚。

图 4 Requester / Completer 不是固定角色，而是“这笔事务里的角色”



很多初学者会本能地以为：

RC 永远是 Requester
EP 永远是 Completer

其实不对。

当 RC 读取 EP 的 BAR 空间时：

RC = Requester
EP = Completer

但如果 EP 自己发起 DMA 去读 Host DDR:

```
EP = Requester
RC / Host Memory 侧 = Completer
```

所以更准确的理解是：

- **RC / EP**：是拓扑角色
- **Requester / Completer**：是某一笔事务里的角色

这两个维度不要混在一起。

1.9 CplID 返回时，系统靠什么找到原始 MRd?

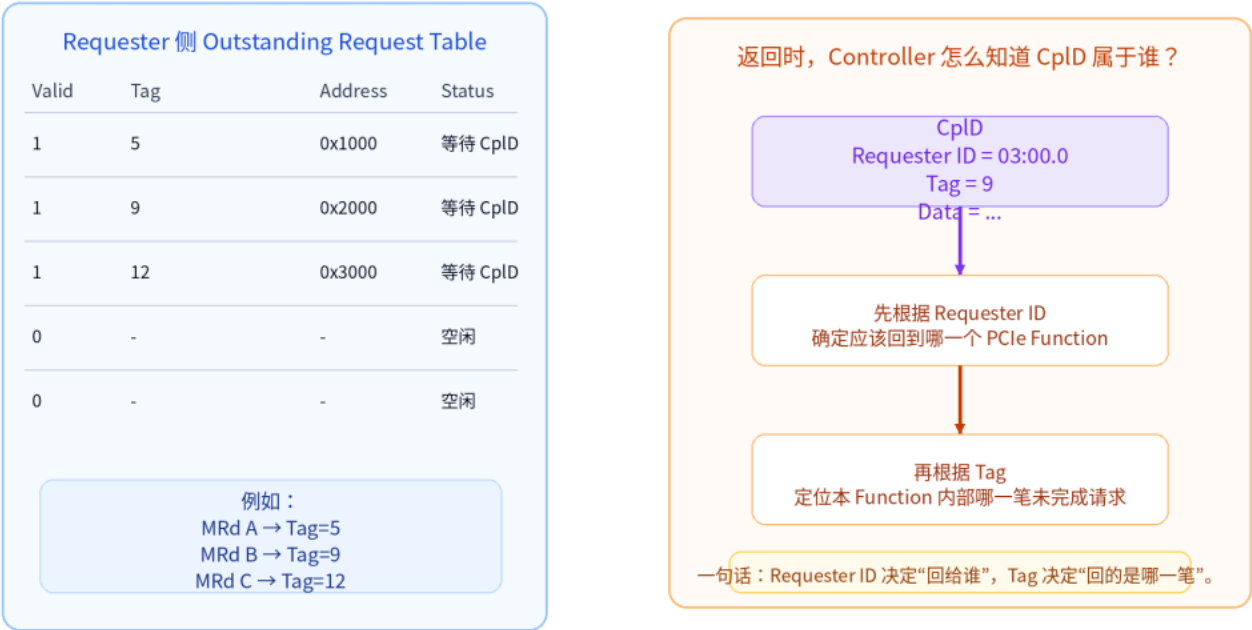
这就是本篇最关键的机制。

假设 RC 同时发出三笔 MRd:

```
MRd A: Tag = 5, Address = 0x1000
MRd B: Tag = 9, Address = 0x2000
MRd C: Tag = 12, Address = 0x3000
```

Controller 内部会把这些未完成请求记录在一张表里，也就是常说的 **Outstanding Request Table**。

图 5 CplD 怎样找到原始请求：Requester ID + Tag



当某个 CplD 返回时，例如：

```
Requester ID = 03:00.0
Tag          = 9
Data         = ...
```

Controller 会先根据 **Requester ID** 确定：

这个 Completion 应该回到哪一个 PCIe Function。

然后再根据 **Tag** 确定：

它对应的是这个 Function 内部哪一笔未完成请求。

一句话概括：

```
Requester ID 决定“回给谁”
Tag          决定“回的是哪一笔”
```

这样，即使多个读请求的返回顺序发生变化，Controller 也不会搞混。

这就是为什么 PCIe 读请求可以乱序返回，却仍然不会把数据交错给错误的事务。

1.10 为什么一笔 MRd 可能对应多个 CplD?

除了“多笔 MRd 可能乱序返回”，还有一种更进一步的情况：

同一笔 MRd，本身就可能被拆成多个 CplD。

例如一笔较大的读请求：

```
MRd
Requester ID = 03:00.0
Tag          = 12
Address      = 0x1080
Length       = 256B
```

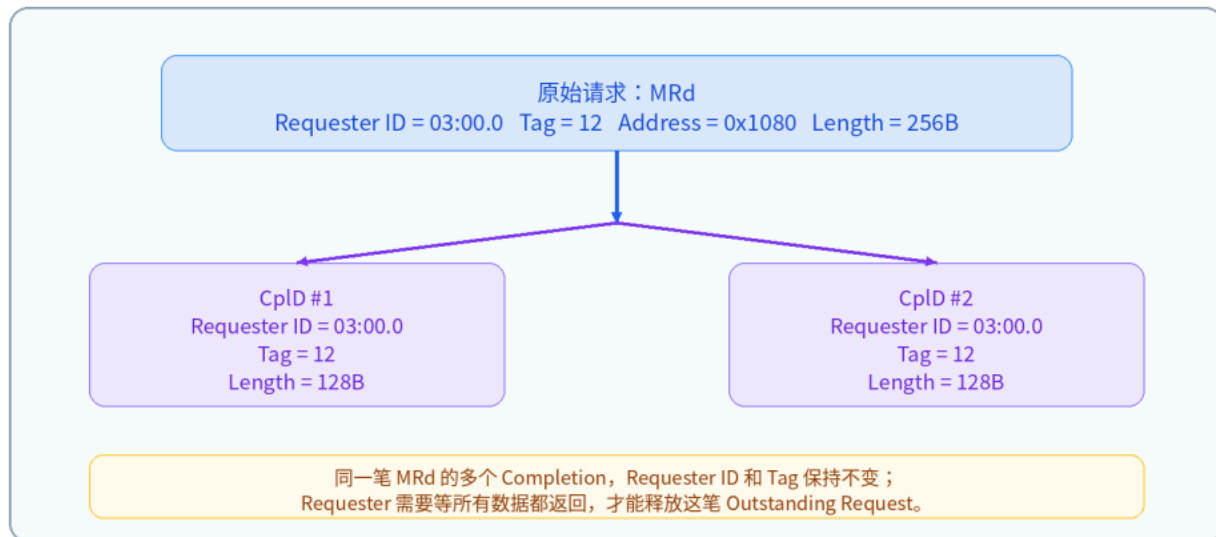
对端未必一次把 256B 全部送回来。

它完全可能拆成：

CplD #1: 128B, Tag = 12

CplD #2: 128B, Tag = 12

图 6 为什么一笔 MRd 可能对应多个 CplD ?



这时有两个重要结论：

1. 同一笔 MRd 拆出来的多个 Completion, **Requester ID 和 Tag 保持不变**。
2. Requester 必须等整笔请求的全部数据都返回，才能真正完成这笔 Outstanding Request。

所以在工程实现里，Tag 不能在“收到第一段数据”后就提前释放。

只有当：

- 所有数据都返回完成，或者
- 这笔请求被明确错误终止

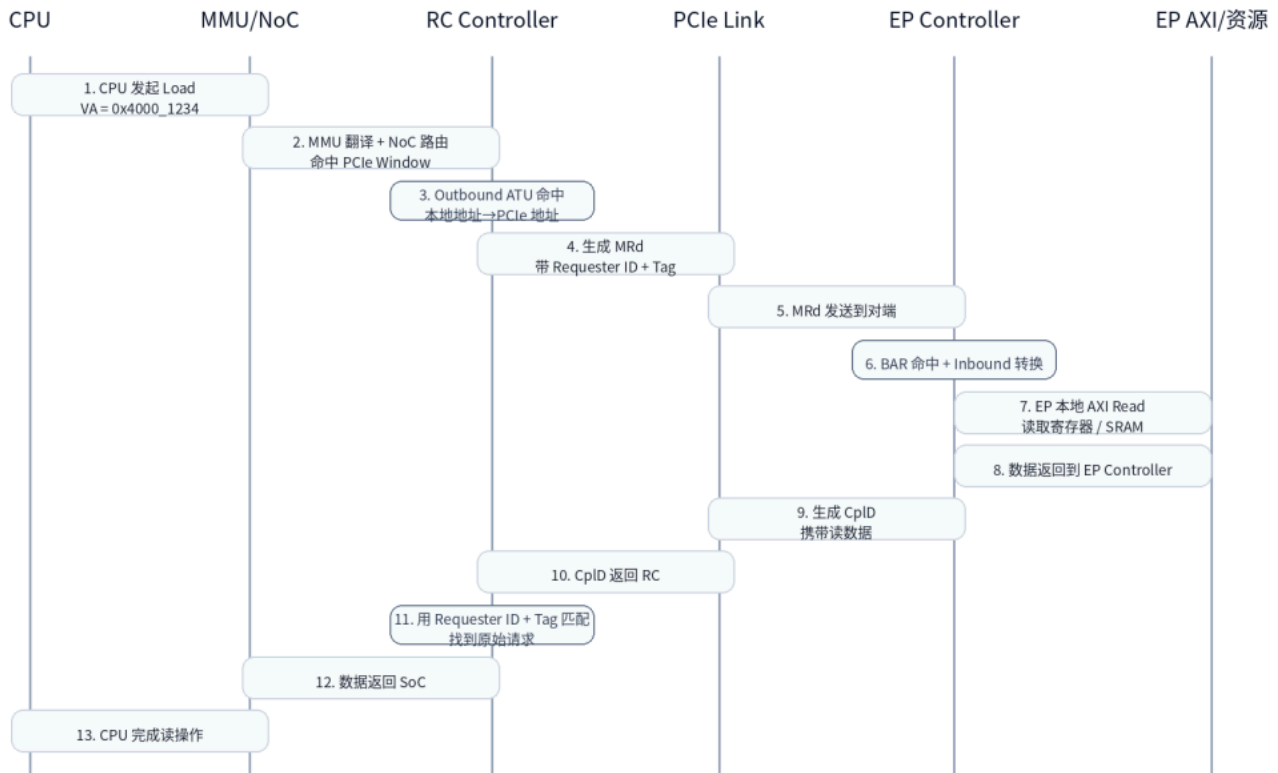
这张 Outstanding 表项才能被清掉。

1.11 最后一步：RC 匹配完成后，数据怎样回到 CPU？

当 CplD 回到 RC 后，Controller 用 Requester ID + Tag 找到原始请求，再把数据重新交还给 SoC 内部的读路径。

于是整体时序可以抽象成下面这样。

图7 一笔 CPU Load 的时序：从 MRd 到 CplD



所以，从 CPU 视角看，最终发生的是：

- CPU 发起 Load
- 系统等待数据
- 数据回来
- CPU / AXI 读完成

而在这个“等待数据”的时间窗口里，背后其实已经完成了：

- 本地地址访问
- PCIe MRd
- 对端 BAR / Inbound / AXI 取数
- CplD 返回
- Requester ID + Tag 匹配

这也正是为什么读路径比写路径对延迟、Outstanding 数量、Tag 资源更敏感。

1.12 为什么 Memory Read 会超时？

一旦理解了整条路径，读超时就不再是“玄学问题”。

它一定是这条链路中某个环节出了问题。

图 8 Debug Checklist：为什么 Memory Read 会超时？



建议固定沿着下面这 7 步查：

1.12.1 CPU / AXI 真的发起了 Read 吗？

先看软件访问、地址、缓存属性、总线是否真的有读事务。

1.12.2 NoC / AXI 是否命中 PCIe Window？

如果地址地图错了，根本不会进入 PCIe。

1.12.3 Outbound ATU 是否命中？

Base / Limit / Target / 类型是否正确。

1.12.4 RC 真的发出了 MRd 吗？

检查：

- TLP Type
- 地址
- Length

- Tag
- Requester ID

1.12.5 Endpoint 是否收到了 MRd，并正确 BAR 命中？

很多问题在这里就丢了。

1.12.6 Endpoint 本地 AXI Read 是否真正完成？

这通常是读路径里最关键的一层。

因为即使链路没问题，如果对端本地寄存器 / SRAM / DDR 响应不了，CpID 也生成不出来。

1.12.7 CpID 是否被正确返回并匹配？

检查：

- Completion Status
- Requester ID
- Tag
- Length
- 数据是否完整

一旦这条链路走通，你就能很清楚地区分：

是“请求没发出去”

还是“请求到对端了但对端没取到数据”

还是“对端拿到数据了但 CpID 没回来”

还是“CpID 回来了但本端没匹配成功”

这才是有工程价值的 PCIe Debug 方法。

1.13 最容易理解错的 6 个点

1.14 误区 1：MRd 发出去时，数据已经在包里了

不是。

MRd 只是在发“读请求”。真正的数据在后续的 CpID 里。

1.15 误区 2: CplD 只靠顺序匹配

不是。

因为多个读请求完全可能乱序返回。

真正匹配依赖的是：

Requester ID + Tag

1.16 误区 3: RC 永远是 Requester

不对。

当 Endpoint 发 DMA 去读 Host 内存时，Requester 就变成了 EP。

1.17 误区 4: CplD 只会有一个

不一定。

大一些的读请求可能被拆成多个 Completion 返回。

1.18 误区 5: 只要链路是 L0，读就一定不会超时

不对。

L0 只说明链路建好了。

真正的数据路径还可能卡在：

- BAR
 - Inbound Translation
 - EP 本地 AXI Read
 - Completion 生成
 - Completion 匹配
-

1.19 误区 6: Memory Read 和 Memory Write 只是 “一个读、一个写” 而已

它们的本质区别比 “方向不同” 大得多：

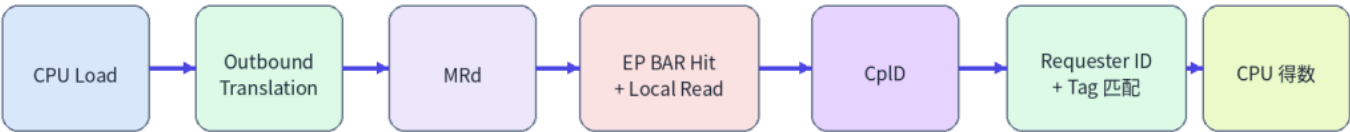
- MWr: Posted, 只发出去即可
- MRd: Non-Posted, 必须等待数据返回

所以很多复杂机制天然围绕 Read 路径展开。

1.20 一张图记住全文

图 9 一张图记住全文

PCIe Memory Read：先发 MRd，再由对端取数并用 CplD 返回；Requester 依靠 Requester ID + Tag 匹配原请求。



读路径比写路径多出来的是完整“返回闭环”：
对端 Local Read → CplD Generate → 返回链路 → Outstanding Match。

如果只保留一句话，那就是：

PCIe Memory Read 的本质，是先发出一笔 MRd 请求，再把数据通过 CplD 送回来；返回时依靠 Requester ID + Tag 找到原始请求。

以后看到：

```
value = *(
volatile
volatile
uint32_t
uint32_t
*)addr;
```

如果 `addr` 落在 PCIe Window，脑子里应该自动展开成：

```
CPU Load
→ NoC / AXI
→ Outbound Translation
→ MRd
→ PCIe Link
→ EP BAR Hit
→ EP Local AXI Read
→ CplD
→ Requester ID + Tag 匹配
→ 数据返回 CPU
```

这就是第二篇最想建立的认知模型。

1.21 后记

技术很重要，技术背后的思想更重要。

理解 PCIe Memory Read，最重要的不是背住几个英文缩写，而是理解这样一个现实事实：

请求和数据并不在同一个动作里完成。

你先把“我要什么数据”告诉对端，再等对端把数据拿回来。

为了在高并发、可乱序的场景下仍然把数据交还给正确的请求，PCIe 才引入了 Requester ID、Tag、Outstanding Request、Completion 这些机制。

下一篇，如果继续顺着这条主线展开，最自然的题目就是：

1.22 《PCIe Tag 到底有什么用？为什么 Read 可以乱序返回？》

下一篇可以专门把：

- Tag 生命周期
- Outstanding Request Table

- 多个 CplD
- Completion Timeout
- Unexpected Completion

彻底讲透。

1.23 参考资料

1. PCI Express Base Specification, Revision 6.0, PCI-SIG。
2. MindShare, *PCI Express System Architecture*。
3. 文中地址与拓扑均为便于理解而构造的示例，不对应某个唯一实现。

知乎专栏：芯片设计进阶之路

微信公众号：芯片进阶之路

2 PCIe Tag 到底有什么用？为什么 Read 可以乱序返回？

来源：<https://mp.weixin.qq.com/s/b3skkqaEeaTnIHafyhJNQA>

作者：桂围玮未

update 2026/08/16 03 : 26

已截图

《PCIe 进阶之路》第 03 篇 · 不背协议，沿着真实数据路径学 PCIe 公众号：芯片进阶之路 · 知乎专栏：芯片设计进阶之路 · 作者：桂围玮未

在前两篇文章里，我们已经把两条最核心的数据路径串起来了：

- 第 1 篇：CPU Store → MWr → 对端 BAR / AXI
- 第 2 篇：CPU Load → MRd → 对端取数 → CplD 返回

到了这里，一个新的问题几乎一定会冒出来：

如果同时发出了很多笔 Memory Read，请求返回时，系统怎么知道每个 CplD 属于哪一笔原始 MRd？

更进一步：

为什么 PCIe 允许 Read 请求乱序返回？这样不会把数据搞乱吗？

这两个问题，正是 **Tag** 存在的原因。

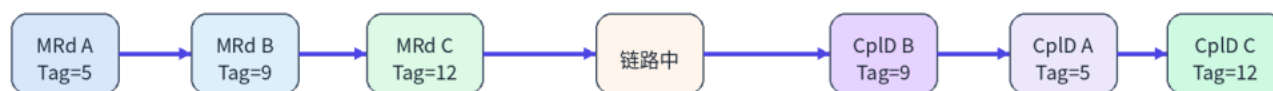
所以第三篇我们不再讲“读请求怎么从 RC 到 EP 再回来”，而是只聚焦一个核心机制：

2.1 Tag 到底在解决什么问题？

PCIe Tag 到底有什么用？

为什么 Read 可以乱序返回？

公众号终稿素材包 · 第三篇 · 芯片进阶之路



Tag 的作用不是“让数据更快”，而是让系统在多笔并发、乱序返回的场景下，仍能准确知道每个 CplD 属于哪一笔请求。

2.2 如果只有一笔 MRd，其实根本感受不到 Tag 的存在

假设系统里永远只有一笔读请求：

```
MRd A
→ 等待
→ CplD A 返回
```

这种情况下，就算没有 Tag，很多人也会本能地觉得：

反正只有一个请求，回来的一定就是它。

这没错。

Tag 真正体现价值的场景，从来不是“只有一笔请求”的世界。

它存在的前提是：

多个 Read Request 并发 Outstanding

也就是说，请求已经发出去了，但还没完成。

例如：

MRd A: 读地址 0x1000
MRd B: 读地址 0x2000
MRd C: 读地址 0x3000

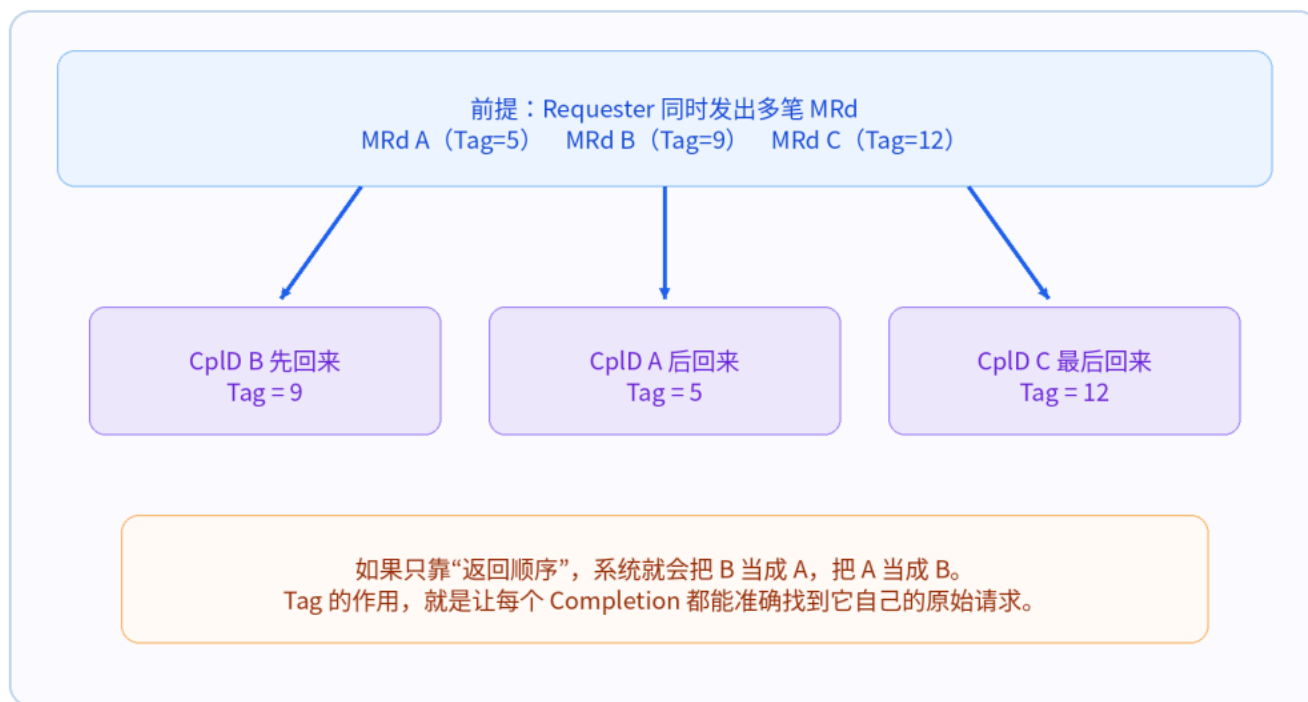
如果这三笔请求几乎同时在路上，那么系统就必须回答：

当某个 Completion 返回时，我怎么知道它属于 A、B 还是 C？

这就是 Tag 的起点。

2.3 先看最直观的问题：为什么不能只靠“返回顺序”？

图 1 第三篇主线：Tag 为什么存在？



很多初学者第一次接触 Completion 匹配时，会下意识地想：

先发的先回，后发的后回，不就行了吗？

如果世界真这么简单，Tag 的重要性就会小很多。

但真实系统不是这样。

同一时间发出的多个 MRd，目标资源的准备时间完全可能不同：

- 某一笔地址已经命中缓存，数据很快准备好
- 某一笔地址需要更长的内部访问路径
- 某一笔地址甚至会被拆成多段返回

因此，谁先返回，本质上取决于：

哪一笔请求对应的数据先准备好。

而不是“谁先发出去”。

所以，PCIe 允许这种情况发生：

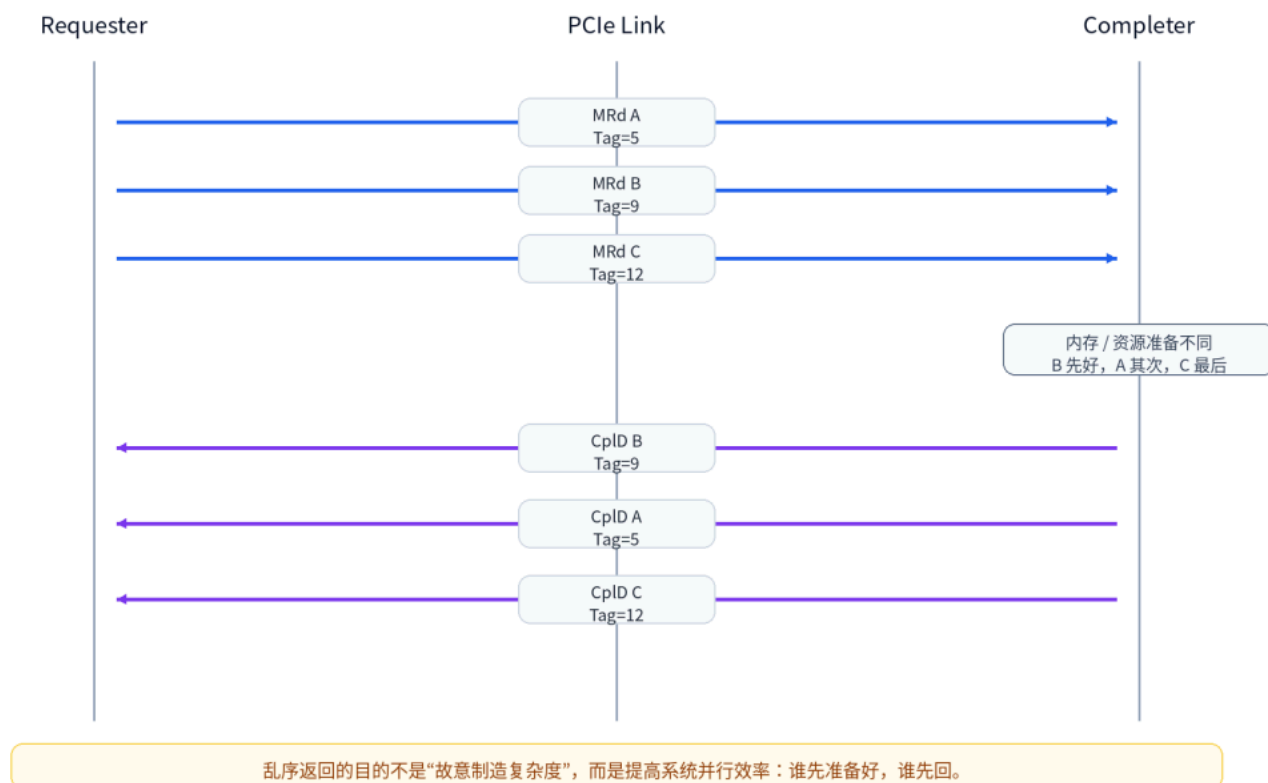
先发 MRd A
再发 MRd B
最后却先收到 Cp1D B，再收到 Cp1D A

如果这时系统只靠“返回顺序”去猜，那么结果一定会错。

2.4 为什么 PCIe 允许 Read 乱序返回？

这个设计并不是为了给实现增加复杂度，而是为了性能。

图 2 为什么 PCIe 允许 Read 乱序返回？



想象下面这个场景：

MRd A → 去读一个较慢资源
MRd B → 去读一个已经准备好的资源
MRd C → 去读另一个普通资源

如果强制规定：

B 就算已经准备好了，也必须等 A 回来之后才能回。

那会发生什么？

- 链路空着
- 完成包明明能回，却被人为压住
- 并发读的价值被浪费
- 整个系统吞吐变差

因此，PCIe 更倾向于：

谁先准备好，谁先回

这就是“Read 可以乱序返回”的本质。

也正因为允许乱序，系统才必须有一种机制，能在返回时准确定位原始请求。

这个机制就是：

Requester ID + Tag

2.5 Tag 的第一层作用：给每一笔未完成请求一个编号

你可以先把 Tag 想象成：

Requester 给每一笔 Outstanding Read Request 发的一张号码牌。

例如：

MRd A → Tag = 5
MRd B → Tag = 9
MRd C → Tag = 12

这样，当 Completion 返回时，它不会只说：

我带回了一些数据

而是会说：

我带回的是 Tag = 9 那笔请求的数据

于是系统就能知道：

这是 MRd B 的 Completion。

这就是 Tag 最朴素、也最重要的作用：

让每个返回的 Completion 都能指回它自己的原始请求。

2.6 光有 Tag 还不够，为什么还要有 Requester ID？

到了这里，另一个问题自然会出现：

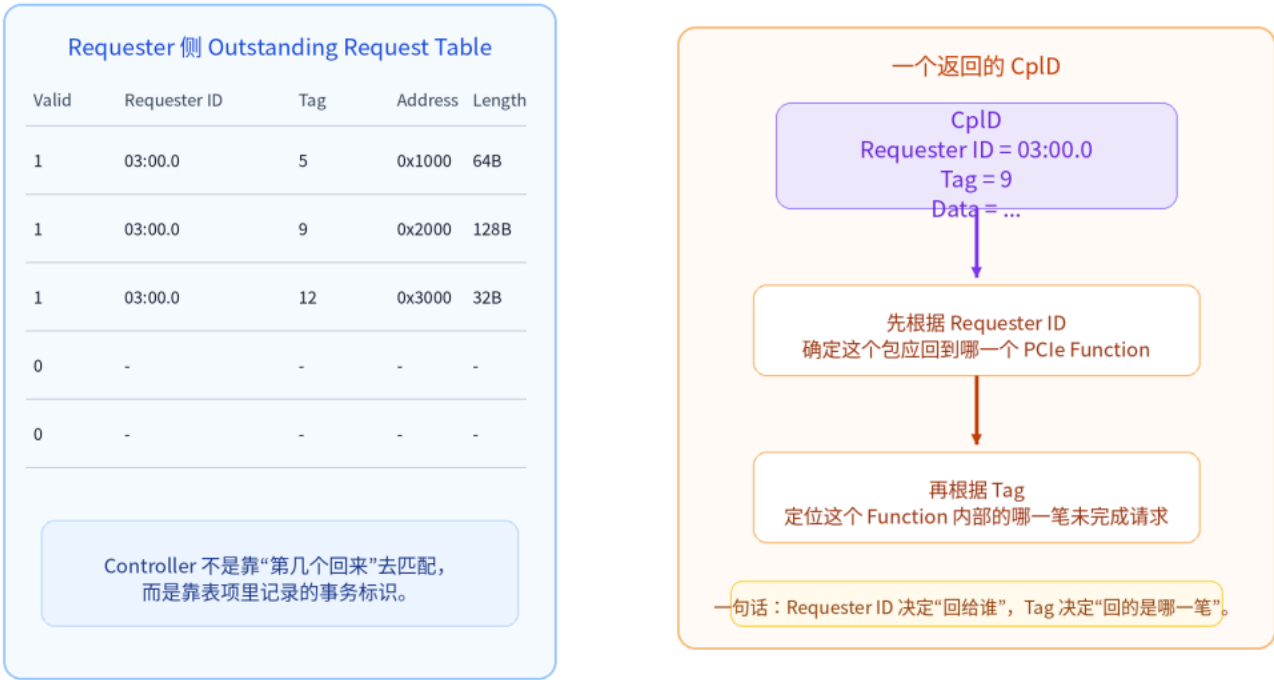
如果两个不同设备都用了 Tag=5，难道不会冲突吗？

答案是：

会重复，但不冲突。

因为 Completion 的匹配从来不只靠 Tag 一项。

图 3 Completion 靠什么匹配原始请求：Requester ID + Tag



真正起作用的是：

Requester ID + Tag

你可以把这两个字段分工理解成：

Requester ID
→ 这包该回到哪一个 PCIe Function
Tag
→ 回到这个 Function 之后，属于哪一笔未完成请求

一句话概括就是：

Requester ID 决定“回给谁”，Tag 决定“回的是哪一笔”。

这就是为什么在协议实现里，我们经常会说：

Transaction ID $\approx$ Requester ID + Tag

它们一起决定了一笔 Completion 的归属。

2.7 Outstanding Request Table: Tag 真正落地的地方

Tag 不是一个空洞的概念，它最终一定会落到某种实现结构里。

最常见的抽象就是：

Outstanding Request Table

也就是“未完成请求表”。

当 Requester 发出一笔新的 MRd 时，通常会做几件事：

1. 找一个空闲 Tag
2. 把这笔请求的信息写进 Outstanding Table
3. 把 MRd 发到 PCIe 链路
4. 等待后续 Completion 返回

表里一般会记录：

- Requester ID
- Tag
- 请求地址
- 请求长度
- 本地目标
- 当前状态

这样，当 CplID 返回时，Controller 才能根据：

Requester ID + Tag

在表中准确找到对应的那一行。

所以，Tag 的意义并不是“一个抽象数字”，而是：

它是 Controller 跟踪 Outstanding Read Request 的关键索引。

2.8 Tag 的第二层作用：支持高并发读

有了 Tag 之后，Requester 就不需要这样工作：

发一笔 MRd
等回来
再发下一笔
再等回来

它可以更激进地工作：

连续发很多笔 MRd
每一笔都分配不同 Tag
把它们都挂进 Outstanding Table
谁先回来就先完成谁

这正是现代 PCIe 高吞吐读路径的基础。

换句话说，Tag 让系统拥有了这样一种能力：

在多个请求同时在途的情况下，仍然能把每个 Completion 正确还给原始请求。

如果没有 Tag，那么系统就会被迫回到极低效的串行模式。

所以可以说：

Tag 并不直接“搬运数据”
但它支撑了高并发、可乱序、可并行的读事务模型

2.9 Tag 的生命周期：从空闲到释放

Tag 不能只是“分配一下就完了”，它本身也有生命周期。

图 4 Tag 的生命周期：从空闲到释放



最重要的规则：
不能收到“第一个 CplD”就释放 Tag，必须等整笔请求真正完成。

否则旧请求的后续 Completion 可能撞上新请求，造成严重错配。

一笔典型的 Read Request，其 Tag 一般会经历：

空闲
→ 分配给一笔 MRd
→ 写入 Outstanding Table
→ 等待一个或多个 Completion
→ 整笔请求完成
→ 释放 Tag

这里最重要的一条规则是：

不能只因为收到“一个 Completion”就立刻释放 Tag。

你必须确认：

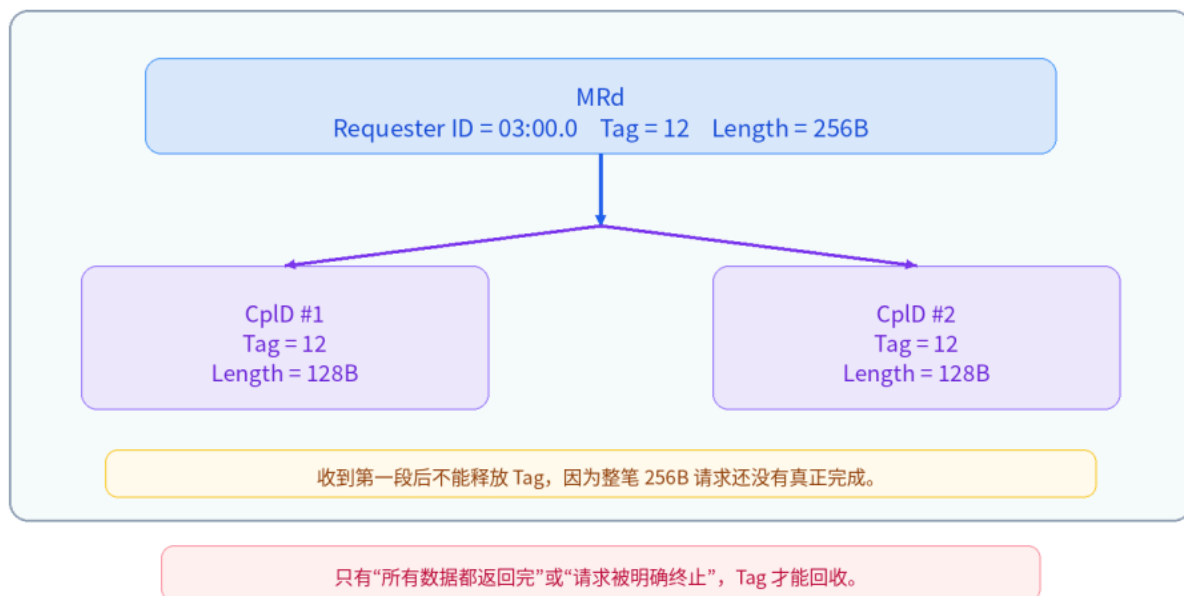
- 这笔请求对应的所有数据都已经返回，或者
- 这笔请求已经被明确地以错误方式终止

否则，过早释放 Tag 会带来严重后果。

2.10 为什么一笔 MRd 可能不止一个 CplD？

这也是很多人第一次接触 PCIe Completion 时会忽略的点。

图 5 一笔请求不止一个 Completion：Tag 为什么不能提前释放



假设：

```
MRd
Requester ID = 03:00.0
Tag          = 12
Length       = 256B
```

这笔请求完全可能被拆成：

```
CplD #1: 128B, Tag = 12
CplD #2: 128B, Tag = 12
```

于是就出现了一个非常关键的实现要求：

收到第一段数据之后，Tag 仍然不能释放。

因为整笔 256B 请求还没有真正完成。

如果你在收到第一个 CplD 时就把 Tag=12 释放掉，并把它重新分给新的请求，那么当旧请求的第二个 CplD 返回时，就会撞上新请求，造成极难调试的错配错误。

因此：

Tag 的释放条件不是“收到过 Completion”，而是“整笔请求已经完成”。

2.11 Tag 满了，是不是 PCIe 出故障了？

这个问题非常经典。

很多人看到 “所有 Tag 都处于 Outstanding” 时，会本能地觉得：

糟了，系统卡死了。

其实不一定。

图 6 Tag 满不一定是故障：正常饱和 vs 真正异常



要分两种情况：

2.12 正常饱和状态

这是完全可能的正常行为：

- 所有可用 Tag 都被已发出的 MRd 占用
- 新的 MRd 暂时不能继续发
- AXI / Request Queue 被正确 backpressure
- 已有 Completion 持续返回
- 老请求完成后，Tag 被释放
- 系统继续前进，只是当前并发度打满了

这种状态不是故障，只是：

Tag 资源暂时用满了。

2.13 真正异常状态

真正需要警惕的是这些情况：

- **Tag 泄漏**：请求完成了，但表项没清掉
- **Tag 提前复用**：旧请求没完成就被新请求占用
- **Unexpected Completion**：回来的 Completion 在表里根本找不到
- **Completion Timeout**：长时间等不到某个 Tag 对应的 Completion
- **没有空闲 Tag，但设计仍错误地接收新的请求**

所以要把：

Tag 满

和：

Tag 管理出错

明确区分开。

2.14 乱序返回 + 多个 Completion + 生命周期，构成了 Tag 的完整意义

到这里，Tag 的作用其实已经完整了。

如果只分条记忆，可以总结成三句话：

2.14.1 Tag 给每笔 Outstanding Read Request 一个编号

这样返回时不会搞不清属于哪一笔。

2.14.2 Tag 让系统能支持多个并发 Read 并允许乱序返回

谁先准备好，谁先回，性能更高。

2.14.3 Tag 必须和 Outstanding Table 一起管理

它不是一个“发出去就忘”的字段，而是本端状态机的一部分。

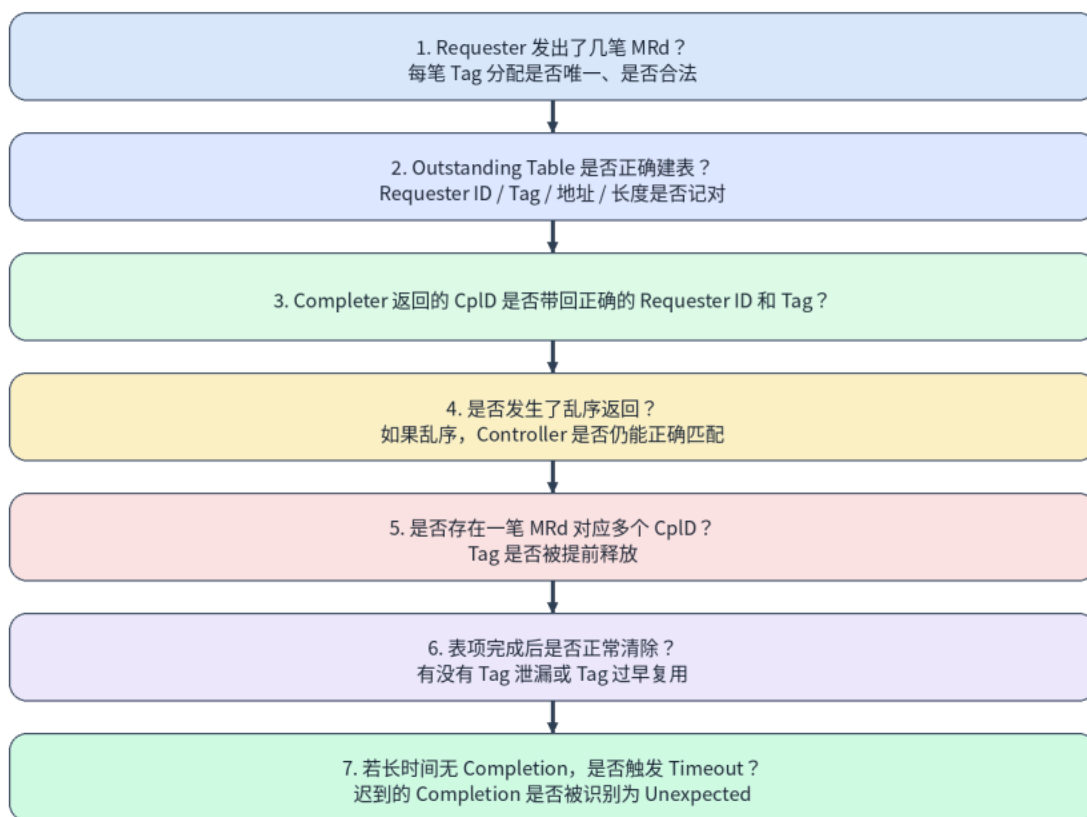
2.15 Debug 时，最应该围绕 Tag 查什么？

如果你的问题已经不再是“链路起不来”，而是这种更偏事务层的现象：

- MRd 发出去了，但 Completion 经常匹配错
- Completion 超时
- 某些请求偶发丢失
- 乱序场景才出问题
- 大包读请求偶发异常

那么你就应该把视角切到 Tag / Outstanding 这一层。

图 7 Debug Checklist：为什么 Completion 匹配会出问题？



建议重点查这 7 个点：

2.15.1 Tag 分配是否唯一？

同一时刻并发 Outstanding 的请求，Tag 是否出现了重复使用。

2.15.2 Outstanding Table 是否正确建表？

Requester ID、Tag、地址、长度是否都正确记录。

2.15.3 返回的 Completion 是否带回了正确的 Requester ID 和 Tag？

这是最基本的对包核查。

2.15.4 如果 Completion 乱序返回，Controller 是否仍能正确匹配？

不要假设返回顺序永远和请求顺序一致。

2.15.5 如果一笔请求被拆成多个 CplID，Tag 是否被提前释放？

这类问题非常常见，也最隐蔽。

2.15.6 请求完成后，表项是否正常清除？

否则就会出现 Tag 泄漏。

2.15.7 超时和迟到 Completion 如何处理？

如果请求已经 Timeout，但迟到的 Completion 仍然回来了，它往往会变成：

Unexpected Completion

这时就必须确认设计是否按预期处理了这种场景。

2.16 最容易理解错的 6 个点

2.17 误区 1：Tag 是为了“加速链路”

不准确。

Tag 不是物理层加速机制，它本质上是事务层的请求跟踪机制。

2.18 误区 2：Tag 在整个 PCIe 系统里必须全局唯一

不对。

不同 Requester 可以使用相同 Tag，只要它们的 Requester ID 不同即可。

2.19 误区 3：乱序返回是一种异常行为

不是。

在满足协议规则的前提下，Read Completion 乱序返回是正常的、甚至是高性能系统希望出现的行为。

2.20 误区 4：只收到一个 Completion，说明这笔请求可以结束了

不一定。

一笔较大的 MRd 完全可能对应多个 CplD。

2.21 误区 5：Tag 满就一定是故障

不对。

如果只是并发度打满、请求在正常流动、Backpressure 也正常，那只是资源暂时用满。

2.22 误区 6：Tag 可以脱离 Outstanding Table 单独理解

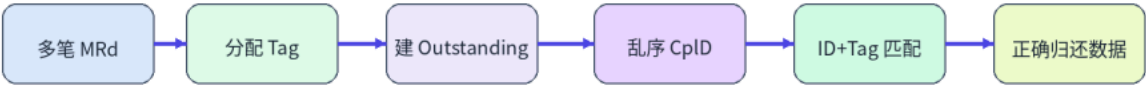
也不对。

Tag 的真实意义，只有放进“未完成请求跟踪”这个上下文里才完整。

2.23 一张图记住全文

图 8 一张图记住全文

Tag 的核心作用：让多个并发、可乱序返回的 Read Request，在 Completion 回来时仍然能准确找到原始请求。



下篇可接：《PCIe Credit 到底是什么？为什么它和 Tag 完全不是一回事？》

如果只保留一句话，那就是：

Tag 的核心作用，是让多个并发、可乱序返回的 Read Request，在 Completion 回来时仍然能准确找到原始请求。

因此，每次看到：

MRd A、MRd B、MRd C 同时在路上

脑子里都应该自动联想到：

- 给每笔请求分配不同 Tag
- 写入 Outstanding Table
- Completion 回来时用 Requester ID + Tag 匹配
- 谁先回来就先完成谁

这就是 PCIe Tag 的本质。

2.24 后记

技术很重要，技术背后的思想更重要。

Tag 之所以值得单独讲一篇，不是因为它本身有多复杂，而是因为它恰好暴露了 PCIe 的一个核心哲学：

协议设计不是为了“看起来整齐”，而是为了在高并发、可乱序、真实复杂的系统里，仍然能准确地跟踪每一笔事务。

理解了 Tag，你就真正迈进了 PCIe 事务层的核心地带。

下一篇如果继续顺着这个脉络展开，最自然的题目就是：

2.25 《PCIe Credit 到底是什么？为什么它和 Tag 完全不是一回事？》

因为接下来最容易混淆的一对概念，正是：

- Tag：跟踪未完成事务
- Credit：反映接收缓冲资源

很多初学者会把这两个东西混在一起。下一篇正好把它们彻底分开。

2.26 参考资料

1. PCI Express Base Specification, Revision 6.0, PCI-SIG。
2. MindShare, *PCI Express System Architecture*。
3. 文中地址、Tag 数字与拓扑均为便于理解而构造的示例。

知乎专栏：芯片设计进阶之路

微信公众号：芯片进阶之路

3 CPU 写一个地址，PCIe 到底发生了什么？

来源：https://mp.weixin.qq.com/s/8_op-4qJpRJUGgVTVjEeLw

作者：桂围玮未

update 2026/08/16 03 : 37

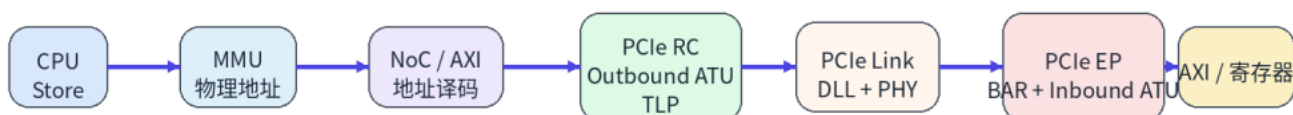
已截图

《PCIe 进阶之路》第 01 篇 · 不背协议，沿着真实数据路径学 PCIe 公众号：芯片进阶之路 · 知乎专栏：芯片设计进阶之路 · 作者：桂围玮未

这篇文章不从 PCIe 三层协议开始背概念，而是只追一笔最普通的 CPU Store：它怎样从 SoC 内部的一次 AXI 写，变成一笔 PCIe Memory Write，再落到另一颗芯片内部的寄存器。

CPU 写一个地址，PCIe 到底发生了什么？

公众号素材包 · 第一篇 · 芯片进阶之路



一条写操作，串起地址映射、TLP 封装、链路发送、BAR 命中和 AXI 访问。

3.1 先问一个很具体的问题

假设软件里有这样一行代码：

```
*(  
volatile  
volatile  
unsigned  
unsigned  
int  
int  
*)  
0x40001234  
0x40001234  
=  
0x55AA  
0x55AA  
;
```

如果 0x4000_1234 这段地址不是 DDR，也不是片上 UART、SPI，而是被系统映射成了一个 PCIe Root Complex 的访问窗口，那么这条看起来再普通不过的 Store 指令，最终可能会让板子另一端一颗 PCIe Endpoint 芯片中的某个寄存器变成 0x55AA。

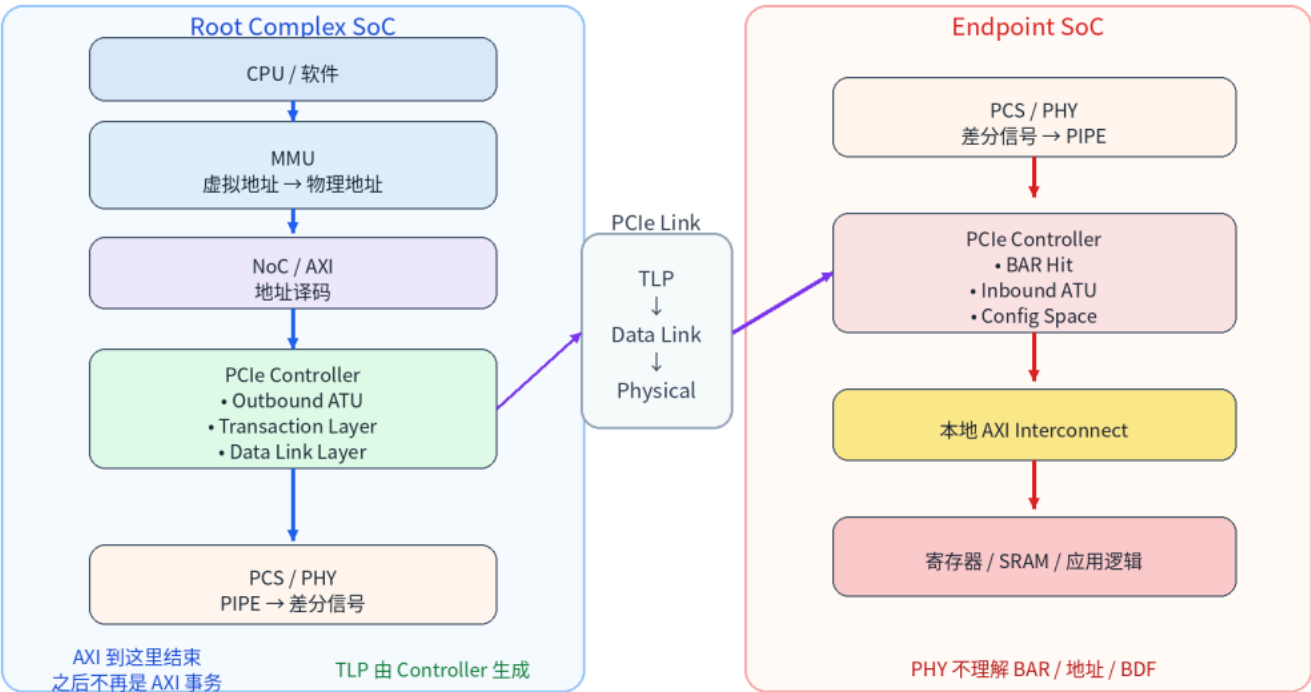
这里真正值得想明白的问题是：

CPU 写的是本地地址，为什么最后能改到另一颗芯片里的寄存器？

很多 PCIe 概念——BAR、ATU、TLP、Memory Write、AXI、Controller、PHY——其实都可以沿着这一个问题串起来。

先看完整路径。

图 1 整体系统地图：从 CPU Store 到对端寄存器



如果先忽略所有协议细节，整条链路可以压缩成：

```
CPU / 软件
  ↓
MMU
  ↓
NoC / AXI
  ↓
PCIe Root Complex
  ↓
Outbound 地址转换
  ↓
Memory Write TLP
  ↓
PCIe Link
  ↓
Endpoint Controller
  ↓
BAR 命中
  ↓
Inbound 地址转换
  ↓
本地 AXI
  ↓
寄存器 / SRAM / 应用逻辑
```

理解这条路径以后，PCIe 就不再是一堆零散名词，而是一套连续的“事务翻译系统”。

3.2 第一步：CPU 发起的其实只是一次本地访存

CPU 本身并不知道“我要发 PCIe”。

它只知道：

我要把数据 0x55AA 写到某个地址。

如果系统开启了 MMU，那么 CPU 最开始使用的可能还是虚拟地址。经过页表翻译后，它得到一个 SoC 物理地址，然后交给片内 NoC / AXI Interconnect。

可以先建立这样一个模型：

```
CPU Virtual Address
  ↓ MMU
SoC Physical Address
  ↓ NoC / Interconnect
DDR / PCIe / UART / SPI / ...
```

真正决定这笔访问去 DDR，还是去 PCIe Controller 的，是 **SoC 地址地图和片内地址译码**。

例如：

```
0x0000_0000 ~ 0x3FFF_FFFF → DDR
0x4000_0000 ~ 0x4000_FFFF → PCIe RC Window
0x5000_0000 ~ 0x5000_FFFF → 其他外设
```

这时 CPU 写：

```
0x4000_1234 = 0x55AA
```

NoC 看见地址落在 PCIe Window，就把这笔 AXI 写送到 PCIe Root Complex。

这里有一个很重要的工程直觉：

PCIe 访问首先仍然是 SoC 内部的一笔地址访问。

所以以后遇到“CPU 写 PCIe 设备没有反应”，不要一上来就查 PHY。

第一件事应该先确认：

这笔地址真的打到 PCIe Controller 了吗？

3.3 第二步：本地地址还不是 PCIe 总线地址

这时 Controller 收到的是一个 SoC 本地地址，例如：

```
0x4000_1234
```

但 Endpoint 在 PCIe 地址空间里对应的地址可能是：

```
0x8000_1234
```

这两个地址显然不是一个东西。

因此，大多数 SoC / PCIe Controller 实现中都会存在某种 **Outbound Address Translation** 机制。不同厂商名字可能不同，常见实现会叫：

- Outbound ATU
- iATU

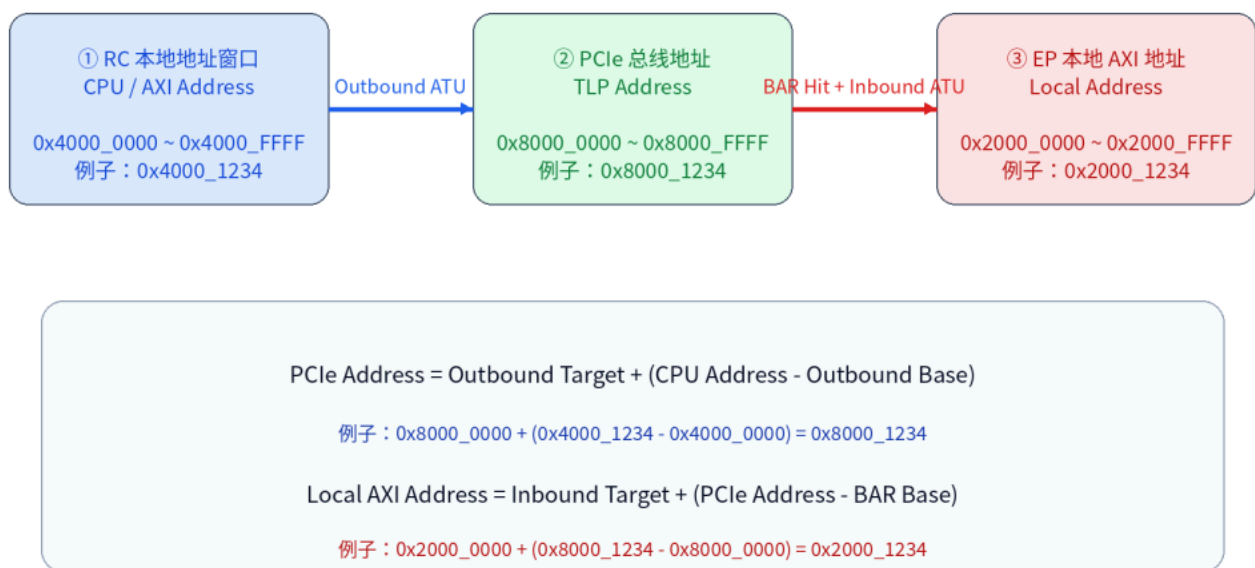
- Address Translation Window
- PCIe Outbound Region

本质都是一回事：

把本地 SoC 地址空间，映射到 PCIe 地址空间。

看一个最简单的例子。

图 3 地址为什么会变三次：CPU 地址、PCIe 地址、EP 本地 AXI 地址



假设：

```
RC 本地 PCIe Window:
0x4000_0000 ~ 0x4000_FFFF
Outbound Target:
0x8000_0000
EP BAR0:
0x8000_0000, 大小 64 KB
EP 本地目标地址:
0x2000_0000
```

CPU 写：

0x4000_1234

Outbound 地址转换后：

```
PCIe Address
= 0x8000_0000
+ (0x4000_1234 - 0x4000_0000)
= 0x8000_1234
```

于是，同一次访问已经出现了两个不同的地址：

```
CPU / AXI Address = 0x4000_1234
PCIe Address      = 0x8000_1234
```

这就是很多初学者理解 PCIe 地址映射时最容易卡住的地方：

CPU 看到的地址，不一定是 TLP 里真正携带的 PCIe 地址。

3.4 第三步：AXI Write 在 Controller 里变成 Memory Write TLP

到这里为止，我们都还在 SoC 内部。

Controller 收到一笔 AXI 写后，需要把它“翻译”成 PCIe 能理解的事务。

对于“CPU 写 Endpoint 的 BAR Memory Space”这个场景，最典型的是：

```
Memory Write TLP
简称：MWr
```

Controller 会根据这笔本地事务，构造出 PCIe Transaction Layer Packet。

一个 MWr 至少需要表达：

- 这是什么类型的事务
- 目标 PCIe 地址
- 数据长度
- Byte Enable
- Payload 数据
- 相关属性

可以粗略理解为：

```
AXI Write
Address = 0x4000_1234
Data    = 0x0000_55AA
      ↓
Outbound Translation
      ↓
PCIe Memory Write TLP
Address = 0x8000_1234
Data    = 0x0000_55AA
```

这里必须建立一个非常清晰的边界：

AXI 到 Controller 为止。

PCIe 链路上不会直接传：

```
AWADDR
AWVALID
WVALID
BRESP
```

这些都是片内 AXI 协议的概念。

Controller 对外送出去的是 PCIe 事务。

3.5 第四步：为什么 Memory Write 不需要 Completion?

这是理解 PCIe 事务类型时非常关键的一点。

普通 Memory Write 属于：

Posted Request

也就是说：

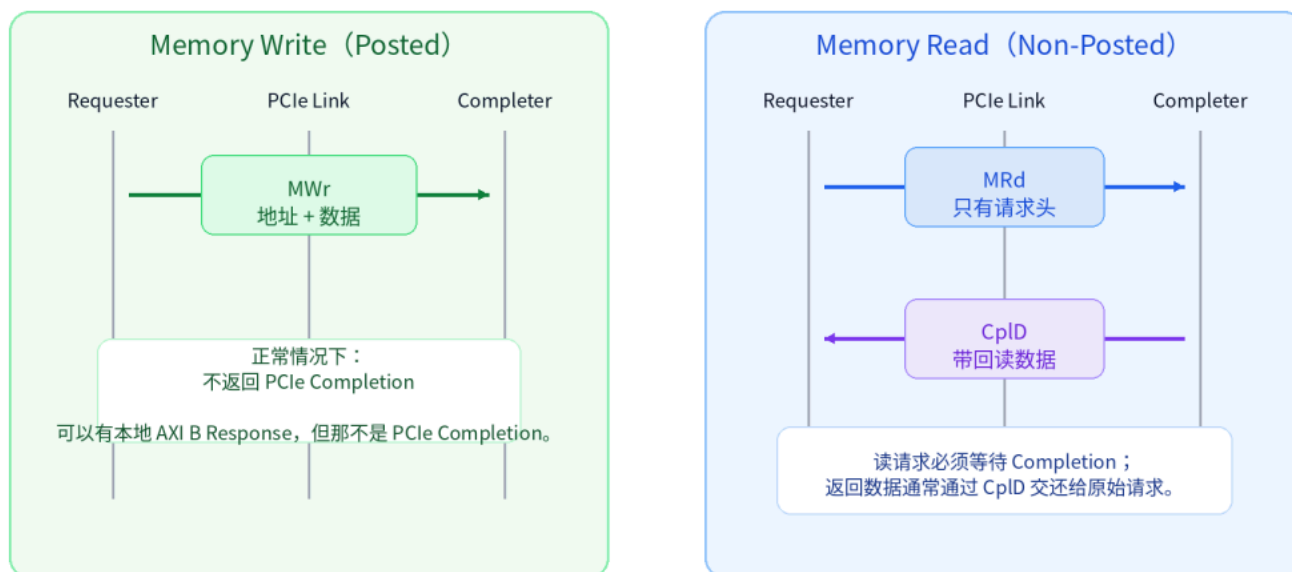
```
Requester ————— MWr —————> Completer
```

正常情况下不会再返回一个 PCIe Completion。

对比 Memory Read：

Requester — MRd —> Completer
Requester ← CplD — Completer

图 5 Memory Write vs Memory Read：为什么写不需要 Completion？



为什么写操作这样设计？

因为如果每一笔 Memory Write 都必须返回 Completion，那么高吞吐写流量会额外产生大量响应包。

PCIe 选择了另一种设计：

Memory Write
→ Posted
→ 正常情况下不返回 Completion

而：

Memory Read
→ Non-Posted
→ 必须返回 Completion with Data

这里再强调一个特别容易混淆的点：

PCIe 没有返回 Completion，不代表 Endpoint 内部没有 AXI B Response。

Endpoint 内部如果最终把这笔写转换成本地 AXI Write，AXI Slave 仍然可能返回 **BRESP**。

但：

```
AXI B Response
≠
PCIe Completion
```

它们属于完全不同的协议层级。

3.6 第五步：PCIe 差分线上到底传的是什么？

TLP 生成以后，还要继续经过 Data Link Layer 和 Physical Layer。

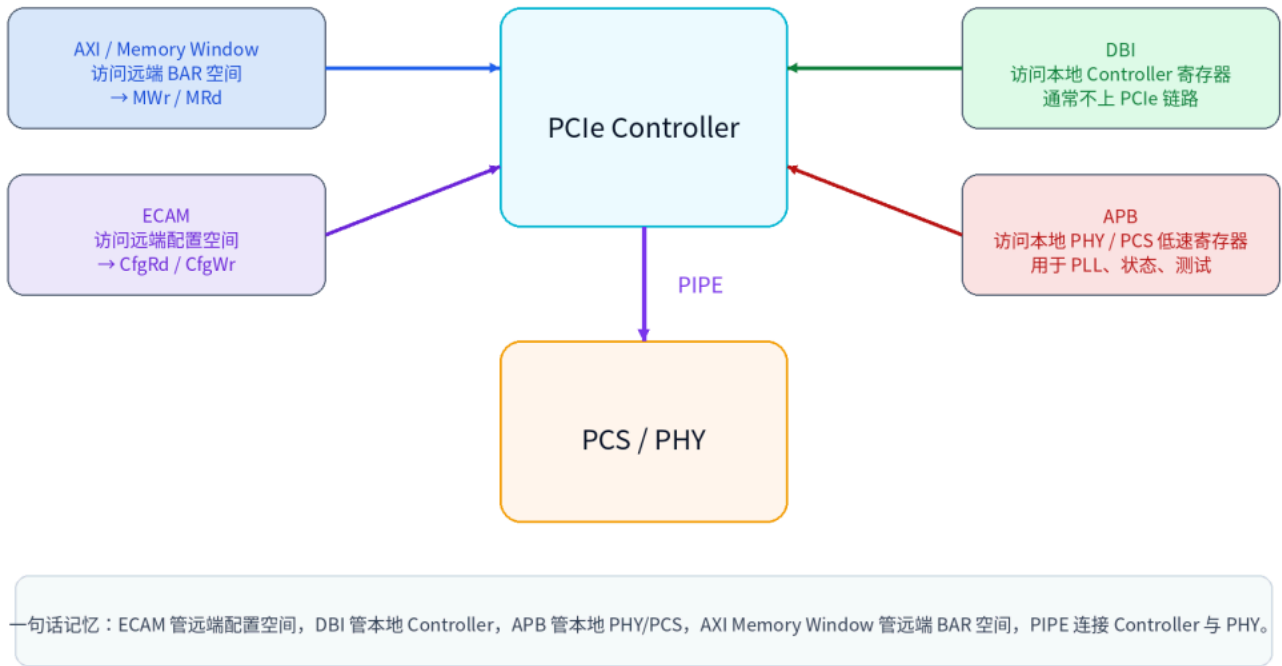
所以真正跨过板级链路的是：

```
TLP
↓
Data Link Layer 处理
↓
Physical Layer 处理
↓
高速差分信号
```

而不是：

- AXI 地址线
- BAR 信号
- APB 地址
- valid / ready
- “直接裸着飞” 的 C 语言地址

图 4 容易混淆的几个接口：AXI / ECAM / DBI / APB / PIPE



对做 SoC 集成的人来说，可以先把几个接口粗暴地分成：

AXI / Memory Window
→ SoC 内部普通数据访问
ECAM
→ 远端 PCIe 配置空间访问
DBI（部分 Controller 厂商常用叫法）
→ 本地 Controller 配置/状态访问
APB
→ 本地 PHY / PCS 等低速配置
PIPE
→ Controller 与 PCS / PHY 之间的专用接口

需要注意：

DBI、APB、ATU 等具体名字和接口形式会因 IP 厂商而不同，但“本地控制面、远端配置面、正常数据面、PHY 接口”这几个逻辑边界是通用的。

3.7 第六步：Endpoint 收到 TLP 后，为什么要先看 BAR？

现在这笔 MWr 已经到了 Endpoint。

TLP 里面带着：

```
PCIe Address = 0x8000_1234
```

Endpoint 不能直接把这个地址送进自己的 AXI。

它首先要判断：

这个地址到底是不是属于我这个 Function?

这里就轮到 BAR 登场了。

假设：

```
BAR0 Base = 0x8000_0000  
BAR0 Size = 64 KB
```

那么：

```
0x8000_1234
```

落在 BAR0 范围内。

于是 Controller 知道：

这是一笔命中 BAR0 的 Memory Request。

接下来再做 Inbound 地址转换。

例如：

```
PCIe Address = 0x8000_1234  
BAR0 Base    = 0x8000_0000
```

得到 BAR 内偏移：

```
Offset = 0x1234
```

如果 Endpoint 本地映射目标是：

```
0x2000_0000
```

那么最终本地 AXI 地址就是：

```
0x2000_1234
```

所以这笔事务完整地经历了：

```
CPU Address   : 0x4000_1234
    ↓
PCIe Address  : 0x8000_1234
    ↓
EP AXI Address: 0x2000_1234
```

这三个地址完全可以不同。

但它们代表的是同一次业务访问。

3.8 最后一步：又变回一笔本地 AXI Write

经过 BAR 命中和 Inbound Translation 后，Endpoint Controller 最终可以在本地侧重新生成一笔 AXI 写。

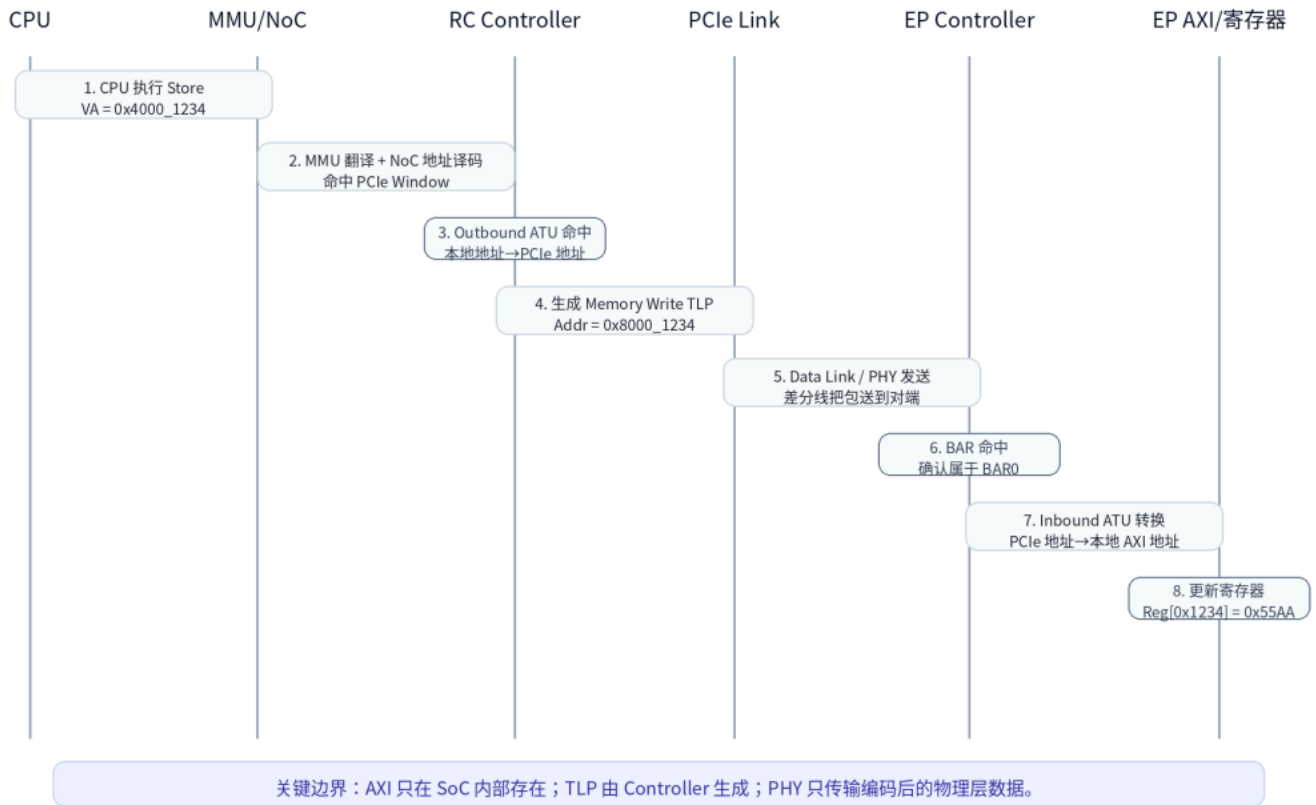
例如：

```
AXI Address = 0x2000_1234
AXI Data    = 0x55AA
```

如果这个地址对应一个控制寄存器，那么寄存器最终被更新。

到这里，一笔 CPU Store 才真正完成了我们关心的数据路径：

图2 一笔 CPU 写操作的时序：从 Store 到对端寄存器更新



把全文压缩成一句话：

CPU 的本地地址访问，被 RC 翻译成 PCIe Memory Write TLP；TLP 经过链路到达 EP 后，再由 BAR 和 Inbound Translation 重新落成本地 AXI 访问。

这就是 PCIe Controller 做的最核心工作之一：

把两个不同芯片内部的本地总线事务，通过 PCIe Transaction 连接起来。

3.9 为什么这条路径对 Debug 特别重要？

因为“CPU 写了但对端没反应”并不是一个问题。

它可能是七个完全不同的问题。

图 6 Debug Checklist：写一个地址却没有生效，应该沿哪条路径查？



建议以后固定按这条路径查：

3.9.1 CPU 真的发起 Store 了吗？

检查：

- 软件地址
- MMU 映射
- Cache / Device 属性
- 写操作是否被优化
- 访问权限

3.9.2 NoC / AXI 真的命中 PCIe Window 了吗？

如果地址地图错了，访问甚至不会进入 PCIe Controller。

3.9.3 Outbound Translation 命中了吗？

重点看：

- Base
- Limit

- Target
- Region Enable
- Memory / Config 类型

3.9.4 Controller 真的生成 MWr 了吗？

检查：

- TLP Type
- Address
- Length
- Byte Enable
- Payload

3.9.5 链路真的把包发出去了么？

这时才开始看：

- Link 是否稳定在 L0
- Replay / Error Counter
- Link 是否反复 Recovery

3.9.6 Endpoint 是否 BAR Hit?

重点看：

- BAR Base
- BAR Size
- Function
- Memory Space Enable
- Inbound Translation

3.9.7 本地 AXI / 寄存器真的写成功了吗？

最后再看：

- AXI 握手
- Slave 是否响应
- 寄存器写保护
- Clock / Reset
- Side Effect

真正高效的 PCIe Debug 方法不是：

猜“可能是 PHY” “可能是 ATU”。

而是：

跟着一笔事务，看它到底消失在哪一级。

3.10 最容易理解错的 5 个地方

3.11 误区 1：AXI 会直接跨芯片传输

不会。

AXI 是 SoC 内部总线协议。

跨 PCIe Link 传的是 PCIe 协议数据。

3.12 误区 2：PHY 知道 BAR 和地址

PHY 不负责理解：

- BAR
- BDF
- Memory Read / Write 语义
- Configuration Space

这些属于 Controller / PCIe 协议逻辑。

PHY 更关心：

- 串并转换
 - CDR
 - Equalization
 - Electrical Idle
 - 差分电气发送与接收
-

3.13 误区 3：BAR 就是一段内存

不准确。

BAR 首先是 Configuration Space 中的 Base Address Register。

它描述/保存的是：

这个 PCIe Function 暴露的一段地址窗口。

真正的 SRAM、寄存器或 DDR，可以位于这段 BAR Window 后面。

3.14 误区 4：MWr 没有 Completion，所以无法判断链路是否可靠

Memory Write 不返回事务层 Completion。

但 PCIe 链路本身仍然有自己的可靠传输机制。

因此：

“没有 Completion”

≠

“链路不做错误检测和重传”

这是事务层和数据链路层职责不同导致的。

3.15 误区 5：写失败优先查 PHY

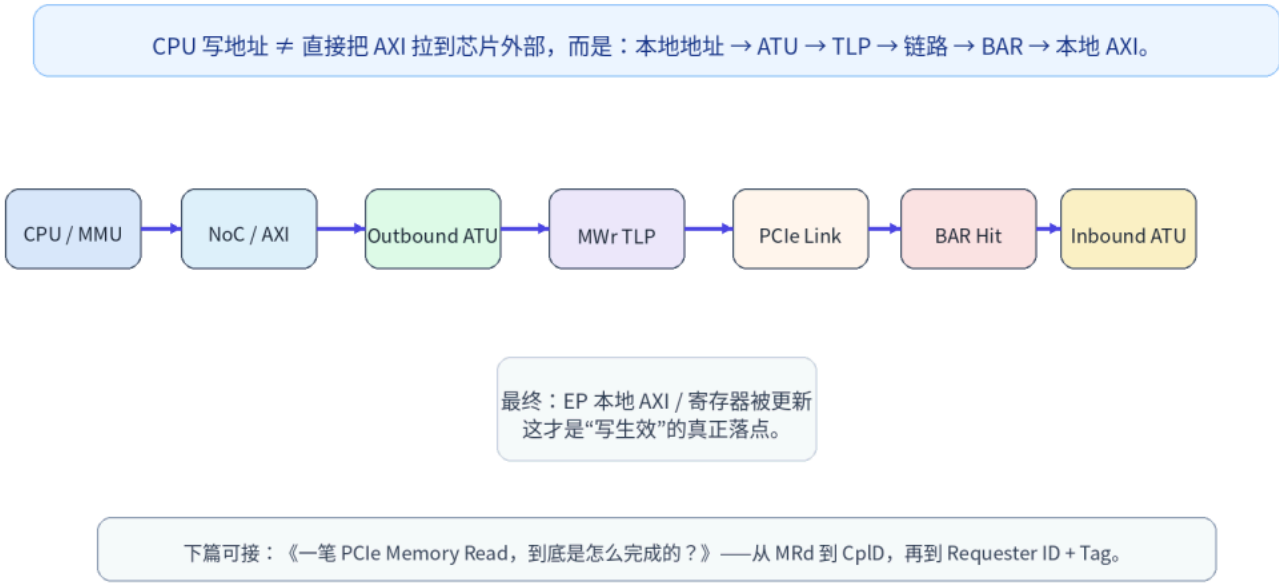
如果链路已经稳定 Link Up，那么大量“访问不生效”问题其实发生在：

Address Map
ATU
TLP Generation
BAR
Inbound Translation
AXI
Register

PHY 只是整条路径的一部分。

3.16 一张图记住全文

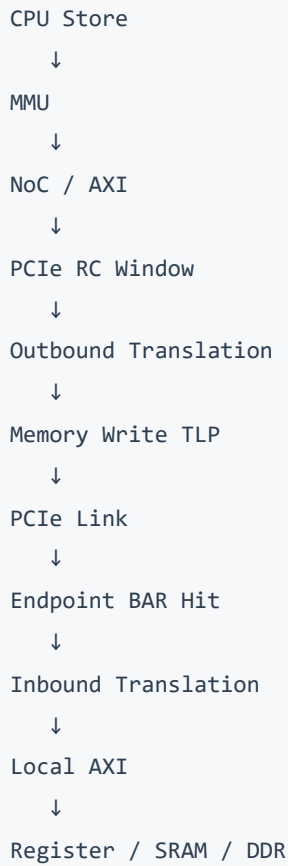
图 7 一张图记住全文



以后看到一句：

```
*(
volatile
volatile
uint32_t
uint32_t
*)addr = data;
```

如果 `addr` 对应 PCIe Window，脑子里应该自动展开成：



这张图比死记几十个 PCIe 名词更重要。

因为后面我们学习：

- BAR
- ATU
- TLP
- Tag
- Credit
- Completion
- LTSSM
- PIPE
- PCS / PHY

其实都只是在逐渐把这条主路径展开。

3.17 后记

技术很重要，技术背后的思想更重要。

PCIe 最值得理解的，并不是“规范里有多少种包”，而是：

不同芯片内部有各自独立的地址空间和总线协议，PCIe Controller 如何把一边的本地事务，转换成一笔可以跨芯片传输的协议事务，再在另一端重新还原成本地访问。

一旦这条主线建立起来，很多原本零散的知识点就会自动找到自己的位置。

下一篇继续沿着这条路径：

3.18 《一笔 PCIe Memory Read，到底是怎么完成的？》

Memory Write 不需要 Completion，但 Memory Read 必须等数据回来。

那么：

- CplD 怎么找到原来的 MRd？
- Requester ID 和 Tag 各自干什么？
- 为什么 Read 可以乱序返回？
- 什么叫 Outstanding Request？
- 为什么一个 MRd 可以对应多个 CplD？

下一篇继续。

3.19 参考资料

1. PCI Express Base Specification, Revision 6.0, PCI-SIG。
2. MindShare, *PCI Express System Architecture*。
3. 本文中的地址、窗口和拓扑均为便于理解而构造的示例，具体 SoC / Controller 实现可能不同。

知乎专栏：芯片设计进阶之路；微信公众号：芯片进阶之路

4. PCIe 为什么不会把接收端 Buffer 撑爆？理解 Credit 流控机制

来源：<https://mp.weixin.qq.com/s/f6B5A62yXr0f3YM1xd4oow>

作者：桂围玮未

update 2026/08/22 07 : 37

已截图

PCIe 链路跑起来以后，TLP 可以一包接一包地往前送。但接收端的 Buffer 明明是有限：**发送端怎么知道什么时候还能继续发，什么时候必须停下来？**

在片上总线里，我们很熟悉 READY / Backpressure：接收端没空间了，就直接把发送端按住。但是 PCIe 是跨芯片的高速串行链路，链路上没有一根像 AXI READY 那样逐拍反馈的信号。

PCIe 换了一种办法：**发送之前，先把“对面还能接多少”这件事算清楚。**

这就是 PCIe 的 **Credit-based Flow Control**。

可以先把它理解成一句话：

Receiver 先告诉 Transmitter 自己能接多少；Transmitter 发 TLP 之前先看额度，额度不够，对应的 TLP 就先不能发。

PCIe Credit 流控的核心关系

Receiver 先广告接收额度，Transmitter 再决定对应 TLP 能不能发送



额度足够 → 可以发送 · 额度不足 → 对应 TLP 暂停

x_chip · PCIe / C2C

FIG 01

4.1 Credit 流控解决的问题

PCIe Flow Control 解决的并不是“这个 Request 最终能不能被 Endpoint 处理”，而是一个更靠近链路的问题：

下一跳 Receiver 有没有足够的接收资源，把这个 TLP 收下来？

这里的“下一跳”很重要。

假设路径是：

Root Port
↓
Switch
↓
Endpoint

Root Port 发给 Switch 时，看的是 **Root Port ↔ Switch 这一跳** 的 Credit。

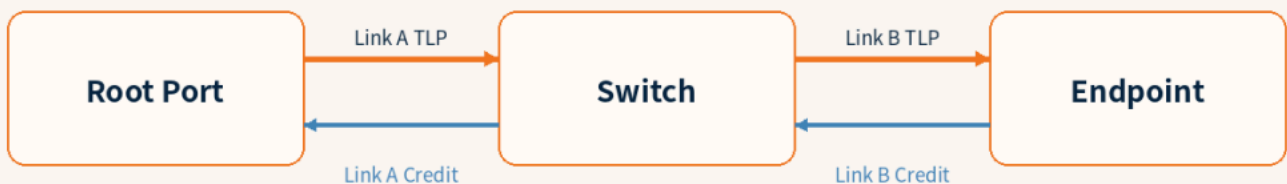
Switch 再往 Endpoint 转发时，要重新看 **Switch ↔ Endpoint 这一跳** 的 Credit。

所以 Credit 本质上是 **Hop-by-Hop** 的接收资源管理，而且两个传输方向各算各的账。它不是一张从 Root Complex 一直管到 Endpoint 的“全局余额表”。

Receiver 把可用接收资源以 Credit 的形式告诉对端，Transmitter 再根据自己维护的 Flow Control 状态决定某类 TLP 现在能不能发。PCIe Base Specification 对这个约束有明确规定：发送 TLP 之前，必须满足相应的 Flow Control Credit 条件。[1]

Credit 是 Hop-by-Hop 的

有 Switch 时，每一跳、每个方向都维护自己的 Flow Control



Link A 有 Credit ≠ Link B 一定有 Credit

Debug 要逐跳定位，不能拿另一条 Link 的 Credit 状态替当前链路做判断

4.2 PCIe 的六类基础 Credit

为了先把核心机制讲清楚，这里先用最常见的六类 Credit 建立基本概念。PCIe 6.0 Flit Mode 下还有 Shared / Merged Flow Control 等扩展，后面再单独讲。

PCIe 的 TLP 并不都是同一类流量。一个 Memory Write、一个 Memory Read Request、一个带数据的 Completion，对接收端资源的占用方式并不一样。

因此 Flow Control 会按事务类别和 Header / Data 分开记账：

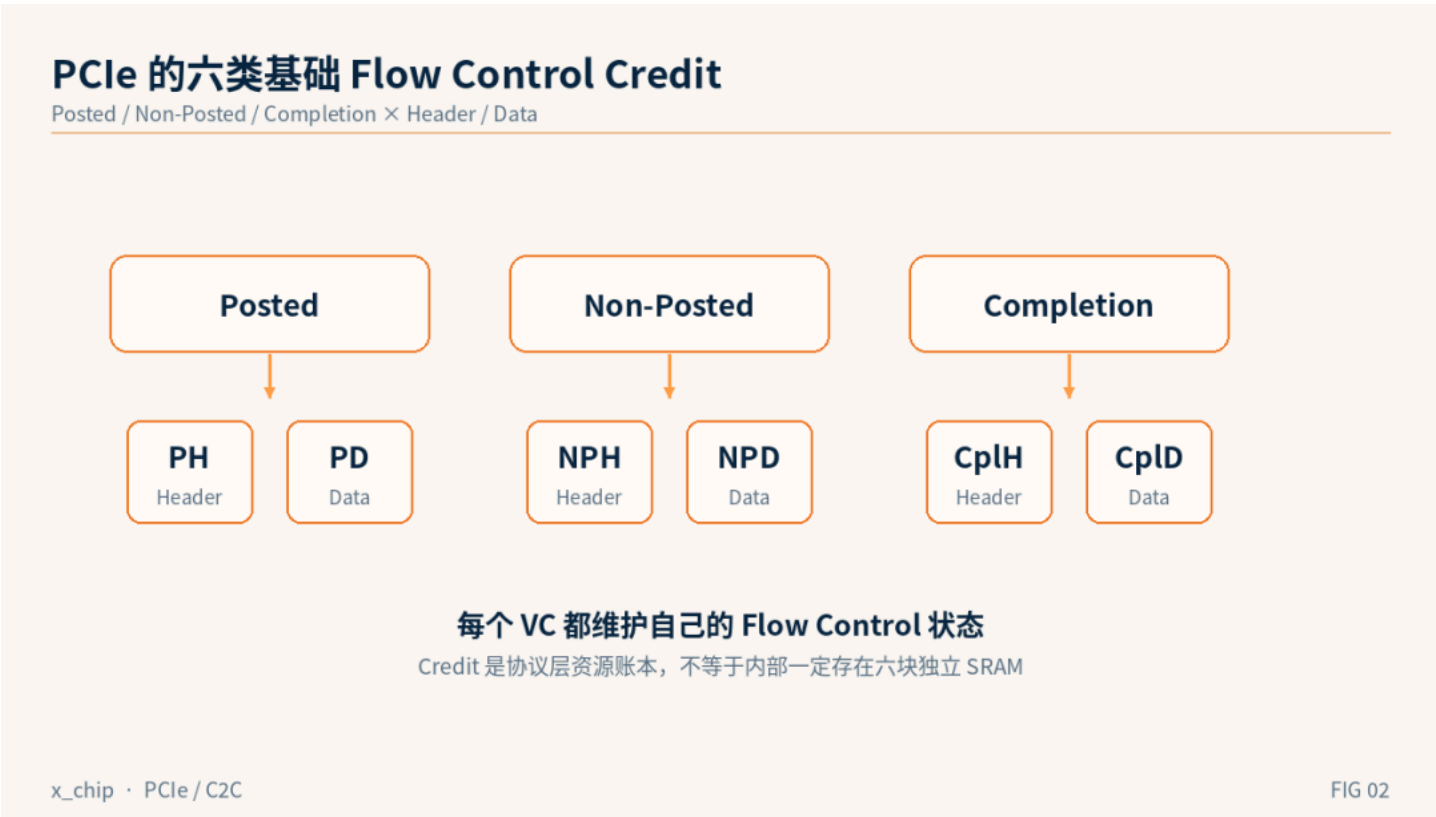
TLP 类别	Header Credit	Data Credit
Posted	PH	PD
Non-Posted	NPH	NPD
Completion	CplH	CplD

这些状态还会按 Virtual Channel 分开维护，于是我们最常见的是：

PH	Posted Header
PD	Posted Data
NPH	Non-Posted Header
NPD	Non-Posted Data
CplH	Completion Header
CplD	Completion Data

这里不要把协议里的六类 Credit 直接理解成“芯片内部一定有六块独立 SRAM”。

Credit 是协议层看到的资源账本；Receiver 内部到底怎么组织 Buffer，是具体实现的问题。



4.3 一笔 TLP 如何消耗 Credit

知道 Credit 的分类以后，判断一笔 TLP 需要什么资源就比较直观了：先看它属于哪一类，再看它有没有 Data Payload。

例如一笔普通 Memory Write：

```
Memory Write
→ Posted Request
→ 需要 Posted Header
→ 有 Payload，还需要 Posted Data
```

可以先记成：

```
MWr → PH + PD
```

再看 Memory Read Request：

```
Memory Read Request
→ Non-Posted Request
→ Request 本身通常没有 Data Payload
```

所以它最核心的 Flow Control 条件是：

```
MRd → NPH
```

等 Completer 把数据返回时，发送的是 **Completion with Data TLP**，这时检查的是 Completion 方向的资源：

```
Completion with Data → CplH + CplD Credit
```

这里先不展开一个具体 TLP 到底消耗多少个 Data Credit Unit。具体数量还和 Payload 大小以及 PCIe 的 Flow Control accounting 规则有关。

这一节真正需要记住的是：

Header 和 Data 分开记账；Posted、Non-Posted、Completion 也分开记账。

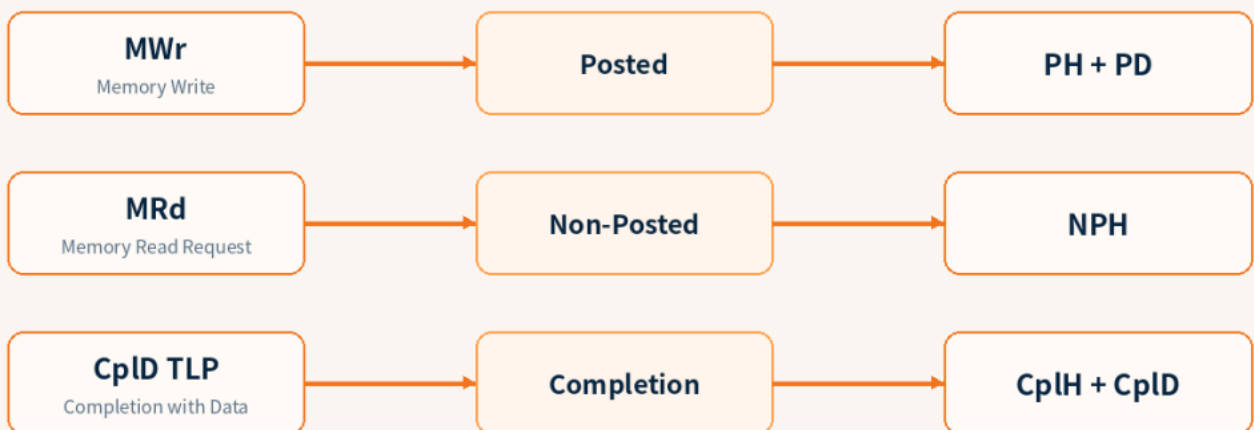
于是再看一笔 PCIe Memory Read，就能看到两个方向其实分别受 Credit 约束：

```
Requester
|
| MRd: 检查 NPH
v
Completer
|
| Completion with Data: 检查 Cp1H / Cp1D
v
Requester
```

如果中间还有 Switch，每一跳又有自己独立的 Flow Control 状态。

不同 TLP 消耗不同类型的 Credit

先判断 TLP 类别，再判断 Header / Data 资源



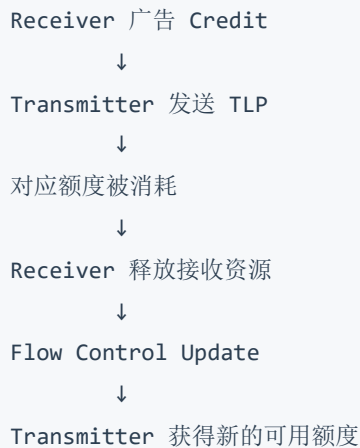
一笔 MRd 的 Forward Request 与返回的 Completion，会分别检查两个方向的 Credit

4.4 Credit 如何建立和更新

链路刚起来时，发送端不能凭空猜对面有多少接收资源。

因此 PCIe 会先完成 Flow Control Initialization，让链路两端建立初始 Credit 状态。之后 Receiver 的接收资源重新可用时，再通过 Flow Control Update 把新的额度信息告诉 Transmitter。

理解上可以把过程压成：



这里有一个很重要的细节：抓到一笔 UpdateFC，并不能简单把里面的数值理解成“Receiver 此刻还剩几个空 Buffer”。

PCIe 使用的是累计式 Flow Control Accounting。发送端结合 Receiver 广告的累计额度和自己已经发送、已经消耗的额度，判断下一笔 TLP 是否还能继续发送。[1]

再往下会涉及 UpdateFC 的计数、回绕以及 Scaled Flow Control。这些可能需要单独写一篇文章讲清楚，这里就不继续深入。

4.5 Credit 与 Tag 的区别

上一篇我们讲过 PCIe Tag。

Credit 和 Tag 经常被放在一起，是因为它们最终都可能限制“还能不能继续增加并发”。但它们管的根本不是同一种资源。

可以先记住这个区别：

Tag 用来标识需要 Completion 的 Outstanding Request; Credit 用来约束下一跳现在还能不能接收对应类型的 TLP。

拿 Memory Read 来看最清楚。

Requester 想增加一笔新的 MRd Outstanding，需要先有可用 Tag。这个 Tag 会跟着 Request 走，直到返回的 Completion 能够找到原来的事务。

但有 Tag 还不够。

真正把 MRd 发到 Link 上之前，还要确认下一跳有足够的 NPH Credit。

于是可能出现两种完全不同的堵法：

Tag 资源耗尽

- Requester 暂时没有新的 Tag 可分配
- 新的 MRd Outstanding 受到限制

和：

NPH 可用额度不足

- Request 已经准备好了
- 但这一跳暂时不允许把 MRd 发出去

Completion 返回时，两者的分工同样清楚。

Completion 里带着用于匹配原始 Request 的 Requester ID / Tag；但这个 Completion 想在反方向 Link 上真正发出来，还要满足那一跳的 CplH / CplD Credit。

Credit 与 Tag：都可能限制并发，但管的不是一件事

Tag 管事务标识，Credit 管下一跳接收资源

	Tag	Credit
核心	标识需要 Completion 的 Outstanding Request	下一跳 Receiver 授予的接收额度
作用范围	Transaction / Completion 匹配	Hop-by-Hop Link Flow Control
受限时	新的 Non-Posted Request 可能无 Tag 可用	对应类型 TLP 暂时不能发送
恢复	原事务完成后 Tag 可再次使用	Receiver 释放资源并更新 Credit

一笔 MRd 往往同时需要：可用 Tag + Forward NPH；返回 CplD 还要满足反向 CplH / CplD

x_chip · PCIe / C2C

FIG 05

所以看到“PCIe 并发上不去”，不能只看到 Tag 或 Credit 其中一个就下结论。

它们最后都可能表现成“新的事务发不出去”，但卡住的位置完全不同。

4.6 可用 Credit 被耗尽时会发生什么

当发送端按自己的 Flow Control 账本判断：某一类 TLP 已经没有足够的可用 Credit 时，这类 TLP 就要先停下来，等待 Receiver 释放资源并更新 Credit。

这里说的“Credit 用完”，是**发送端计算后的可用额度不足**，不是简单看某个 InitFC / UpdateFC 字段的原始数值是不是 0。

这种情况完全可能只是正常的 Backpressure，并不等于：

```
Link Down
PHY Error
整个 PCIe 都停了
```

例如某个 VC 的 NPH 暂时不足，需要 NPH 的 TLP 会被限制；但其他 Credit 类型如果还有额度，相应流量仍可能继续推进。哪些 TLP 能绕过当前被阻塞的 TLP，还要继续满足 PCIe 的 Ordering 和 Deadlock Avoidance 规则。[1]

所以 Debug 时看到某类 TLP 因 Credit 发不出去，第一反应不应该是“Credit 计数器坏了”，而是继续确认：

```
是哪一种 Credit？
属于哪个 VC？
什么时候开始没有新的可用额度？
Receiver 的对应资源有没有继续释放？
Flow Control Update 有没有正常回来？
```

Credit 也不仅关系到“会不会把 Buffer 撑爆”，还会影响吞吐。接收端提供的 Credit 太少，发送端就可能频繁等额度更新，链路 Pipeline 也就很难持续跑满。[1]

4.7 一笔 TLP 发不出去时怎么查

假设现在看到：

```
Link = L0
Requester 有新的 Memory Read 要发
但线上迟迟看不到新的 MRd TLP
```

这时候可以顺着发送路径往前查。

4.7.1 先确认 Requester 真的生成了 Request

如果 Requester 内部根本没有新的事务，查 Credit 没意义。

先确认：

```
Request valid / pending
Tag available
内部 Queue 没被其他条件卡住
```

4.7.2 再确认这笔 TLP 需要哪类 Credit

MRd 属于 Non-Posted Request，所以首先关注：

NPH

不要看到一个 `Credit` 信号就把所有类型混在一起。

4.7.3 检查 Flow Control 是否已经初始化完成

如果初始 Flow Control 状态都没有建立，后面的 Credit 判断本身就没有可靠基础。

4.7.4 检查 Credit 是否继续更新

如果 NPH 长时间处于不可发送状态，再往 Receiver 方向追：

Receiver 对应资源是否真的被占住？
TLP 有没有被后级及时取走？
FC Update 是否正常生成？
Update 是否正常到达对端？
发送端的 Credit Accounting 是否正确？

4.7.5 有 Switch 时逐跳检查

Endpoint 前面的那条 Link 有 Credit，不代表 Root Port 到 Switch 的这一跳也有；反过来也一样。

Credit 是 Hop-by-Hop 的，所以 Debug 也应该 Hop-by-Hop。

Credit 阻塞的 Debug 路径

先确认 Request 存在，再沿“类型 → VC → FC → Receiver”逐级定位



“可用 Credit 耗尽”可以是正常 Backpressure，不等于 Link Down / PHY Error

有 Switch 时，同样的检查需要在每一跳分别完成

x_chip · PCIe / C2C

FIG 06

最后再回头看 Tag。

如果实现里的可观察状态是：

Tag available
NPH unavailable

优先沿 Credit / Flow Control 路径查。

反过来如果：

NPH available
Tag exhausted

那就不是 Flow Control 在拦，而是 Requester 自己的 Outstanding Transaction 资源先到头了。

两种情况在系统层都可能表现成“Read 发不出去”。真正有效的 Debug，还是找到第一处不再往前推进的边界。

4.8 PCIe 6.0 Flit Mode 的边界

前面用六类基础 Credit 建立的是比较理解的基本模型。

到了 PCIe 6.0 Flit Mode，Flow Control 的资源组织和记账方式会继续扩展，例如 Shared Flow Control Credit，以及把部分资源池合并使用的 Merged Flow Control。[2]

这些机制会改变 “Credit 怎么组织、怎么共享”，但核心逻辑并没有变：

发送端在发送 TLP 前，仍然要确认下一跳 Receiver 已经提供了足够的 Flow Control 资源。

所以看到 PCIe 6.0 里更多的 Credit 字段和模式时，可以先记住这个本质。

复杂的是账本怎么扩展，不是为什么需要这本账。

4.9 一张图记住 Credit

把上面内容压缩成一条主线：

```
Receiver 接收资源有限
  ↓
Receiver 广告 Credit
  ↓
Transmitter 按 TLP 类型检查额度
  ↓
额度足够 → 发送
额度不足 → 对应 TLP 等待
  ↓
Receiver 释放资源
  ↓
Flow Control Update
  ↓
继续发送
```

再把 Tag 放回来：

```
Tag
= Outstanding Request 的事务标识
Credit
= 下一跳 Receiver 授予的接收额度
```

一笔 Memory Read 想顺利跑起来，可能同时受到这些资源限制：

```
Requester 有可用 Tag
+
Forward Link 有 NPH Credit
+
Return Link 有 CplH / CplD Credit
```

所以我更愿意把 PCIe Credit 理解成一种 **链路上的接收资源预算**。

它不是用来标识事务，也不是一张端到端的全局 Buffer 表。它做的事情很朴素：

先确认下一跳接得住，再把包发过去。

下一篇继续沿 PCIe 数据链路往下看：**为什么 PCIe 需要 Replay / ACK / NAK，而 Completion Timeout 又为什么不是同一层的问题。**

技术很重要，技术背后的思想更重要！

芯片设计中的很多问题，单看一个模块并不复杂，但放进完整系统后才会真正体现难度。

如果这篇文章对你的工作或学习有帮助，欢迎点赞、在看，也欢迎关注 x_chip，后续一起讨论更多 SoC 架构与芯片设计实践。

4.10 参考资料

[1] PCI-SIG, *PCI Express® Base Specification Revision 6.0.1*, Flow Control / Receive Buffer / DLLP 相关章节。

[2] PCI-SIG, *PCI Express® 6.0 Specification Functionality Updates – Part 2*, Shared Flow Control / Merged Flow Control 相关说明。

转载授权：欢迎全文转载，无需授权；请保留作者「娃围玮末」及来源「微信公众号：芯片设计进阶之路 (x_chip)」，不得冒充原创或歪曲原意。